

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

Autumn 2026

All work for this course can be found in [this github repository](#). There I saved code snippets and other artifacts related to this course. It might be easier to read corresponding `md` files rather than a long pdf submitted for the assignment. Working on assignments for this course I initially wrote answers in my own words and before committing the changes I used Claude to verify my answers and improve wording and grammar, and to unify the formatting across all weeks.

Week 1

1. Define foundational networking terms. Write 2–3 sentences for each term. State what it means, why it matters, and, where useful, give one concrete example or relationship to another term.

Performance and protocol data:

- **network bandwidth** — a parameter that shows how much data can be transmitted.
 - Ever-growing need
 - Limited availability
 - Smart design and usage saves money
 - One important factor to estimate is "how good" the network actually is
- **network throughput** — Actual data successfully delivered over the link
- **packet loss and jitter** — during the data transmission, packets can be lost. It can be referred to as *packet loss*. Even a packet loss rate of 5% can severely reduce effective throughput, depending on the network conditions and communication protocol.
Jitter describes the variation in packet delay or round-trip time (RTT). High jitter can lead to interruptions and reduced performance in communication protocols, especially for real-time applications.

- **bps versus Bps:** *bps* — bits per second (b/s), *Bps* — byte per second (B/s). 1 Bps = 8 bps.

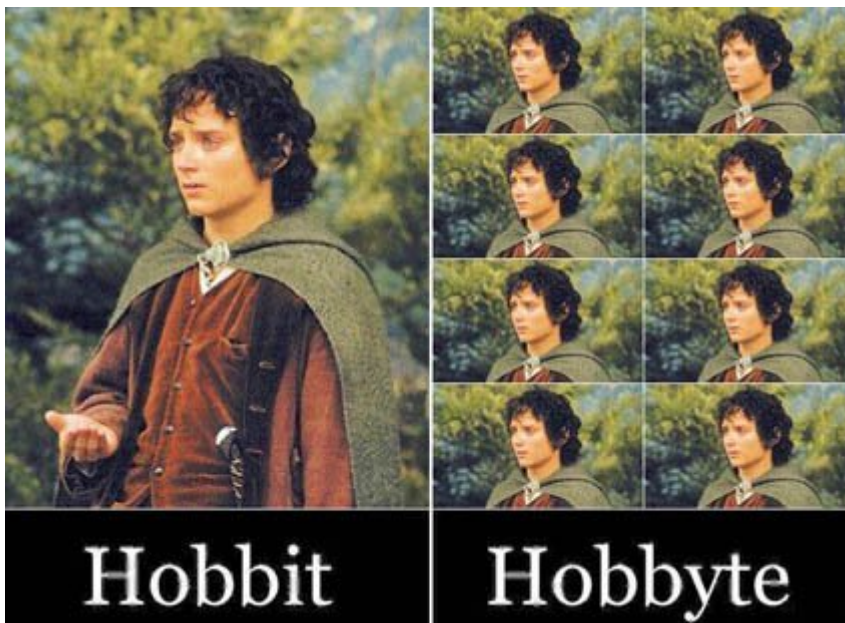


Figure 1.1 — Hobbit/hobbyte joke. Credit to unknown reddit user

- **protocol payload** — the actual useful data a packet carries, for example the sensor reading or the file contents, without any headers. Every protocol layer treats the whole packet from the layer above as its payload. For example, a TCP segment is the payload of an IP packet.
- **protocol overhead, especially in resource-constrained IoT systems** — everything that has to be sent besides the payload: headers, trailers, handshakes, acknowledgements and retransmissions. For a 4-byte temperature reading sent over TCP/IPv6/Ethernet, the headers alone are about 80 bytes, so most of the transmitted data is overhead. On battery-powered IoT devices with slow radios, every extra byte costs energy and airtime. That's why IoT uses compact protocols like MQTT, CoAP over UDP and 6LoWPAN header compression.

Link layer, topology, media, and wireless networking:

- **Spanning Tree Protocol (STP)** — A Layer 2 protocol that prevents switching loops in Ethernet networks by blocking redundant paths. If an active link fails, a blocked path can be activated.
- **collision domain** — A network segment where devices share the same transmission medium and simultaneous transmissions can cause collisions. Switches reduce collision domains by giving each port its own domain.
- **broadcast domain** — A group of devices that receive the same Layer 2 broadcast traffic. Routers separate broadcast domains.
- **SOHO network** — A Small Office/Home Office network designed for a small number of users and devices. It typically consists of a router, Wi-Fi access point, and computers.
- **MAC (physical) address** — A unique Layer 2 hardware address assigned to a network interface. It is used to identify devices within a local network.
- **physical-layer protocol data unit (PDU)** — The PDU of the Physical Layer is a bit. The Physical Layer transmits raw bits over the transmission medium.

- **MAC-layer protocol data unit (PDU)** — The PDU of the Data Link (MAC) Layer is a frame. Frames contain MAC addresses and the encapsulated network-layer data.
- **half-duplex versus full-duplex** — Half-duplex communication allows sending or receiving, but not both at the same time. Full-duplex communication allows simultaneous transmission and reception.
- **Ethernet auto-negotiation** — A process where Ethernet devices automatically determine the highest compatible speed and duplex mode. This occurs when a connection is established.
- **hidden-node problem in wireless networking** — In wireless networks, two devices may be unable to hear each other but can both communicate with an access point. This can lead to collisions and reduced performance.
- **physical versus logical network topology** — Physical topology describes how devices are physically connected by cables or wireless links. Logical topology describes how data flows between devices.
- **TIA/EIA-568 and ISO/IEC 11801** — These are cabling standards that define requirements for structured network cabling, including wiring schemes, performance categories, and installation practices.
- **Ethernet cabling categories, for example Category 6** — Ethernet categories define cable performance characteristics such as bandwidth and supported data rates. Higher categories generally support higher speeds and frequencies. **Cat6** specs: max frequency: 250MHz, typical speed 1Gbps up to 100m, supports 10 Gbps for shorter distances (about 55m)
- **8P8C connector, often informally called RJ45** — An 8-position, 8-contact connector commonly used for Ethernet cables. Although often called RJ45, the technically correct term is 8P8C.
- **Wi-Fi ad hoc mode** — A wireless networking mode where devices communicate directly with each other without an access point. It creates a peer-to-peer network.
- **IEEE 802.11ac, 802.11ax, and 802.11be.** — These are Wi-Fi standards corresponding to Wi-Fi 5, Wi-Fi 6/6E, and Wi-Fi 7 respectively. Each generation improves speed, efficiency, and support for multiple devices.

Tracking the data path

In Windows we can track the connection with `tracert` command. Following shows path from my laptop connected to `Eduroam` network to Google public DNS server. It goes from the local network (everything with a small delay) to Vaasa - Turku - Stockholm

```
tracert 8.8.8.8
```

```
C:\git\studies\componentes_IOT_app main # ?1 ~1 tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

 1      2 ms      2 ms      2 ms    130.231.160.1
 2      2 ms      2 ms      2 ms    at-ext-bb.oulu.fi [193.167.221.66]
 3      3 ms      3 ms      3 ms    oulu-r1.oulu.fi [193.167.221.3]
 4      2 ms      2 ms      3 ms    oulu1-et-0-1-2-2.ip.funet.fi [193.167.244.240]
 5     15 ms     15 ms     15 ms    oulu4.ip.funet.fi [86.50.255.117]
 6     15 ms     14 ms     14 ms    vaasa4-et-0-1-0-1.ip.funet.fi [86.50.254.230]
 7     15 ms     14 ms     15 ms    turku4-et-0-1-0-1.ip.funet.fi [86.50.254.236]
 8     15 ms     15 ms     14 ms    stockholm4-et-0-0-0-1.ip.funet.fi [86.50.254.251]
 9     15 ms     15 ms     17 ms    se-sthb1.nordu.net [109.105.102.0]
10     14 ms     14 ms     14 ms    se-bma2.nordu.net [109.105.101.93]
11      *        *        *      Request timed out.
12     15 ms     15 ms     15 ms    72.14.237.55
13     16 ms     16 ms     16 ms    209.85.242.83
14     30 ms     14 ms     15 ms    dns.google [8.8.8.8]

Trace complete.
```

Figure 1.2 — Result of `tracert` command.

2. Estimate how long does it take to download 3 TB file from cloud based backup service if network download throughput is 200 Mbps for actual payload (i.e. data)?

$\text{file_size} = 3 \text{ TB} = 3,000 \text{ GB} = 3,000,000 \text{ MB} = 24,000,000 \text{ Mb}$

$\text{download_speed} = 200 \text{ Mbps}$

$\text{download_time} = \text{file_size} / \text{download_speed} = 24,000,000 / 200 \text{ s} = 120,000 \text{ s}$

$1 \text{ h} = 3600 \text{ s}$

$\text{download_time_hours} = 33.33 \text{ h}$

3. Locate the MAC address of your mobile phone, laptop wifi interface or some other networked IT device

I have checked MAC addresses of my laptop using following command:

```
# on Windows  
ipconfig /all
```

On my phone I have checked the MAC address from the Wi-Fi settings.

I also verified my findings from my Wi-Fi router's control panel (DHCP Clients List). There you can see all devices currently connected to the network, their `Client name`, `Mac Address`, `Assigned IP`, and `Lease time`. To my surprise, the MAC address of my phone was different from the one I found in the settings. Apparently, it does some MAC randomization.

I won't add a screenshot for privacy reasons, but here are masked MAC addresses of my devices:

- Laptop: 88:D8:2E:00:00:00
- Phone:
 - 7C:7B:BF:00:00:00- actual address from phone settings
 - D6:8F:74:00:00:00 - from the router

Using [this](#) tool I have searched for OUI of my devices. Laptop showed as `Intel Corporate`, actual phone address `Samsung Electronics Co., Ltd`, and the random address that my phone actually used to connect to the router had no matches in this database.

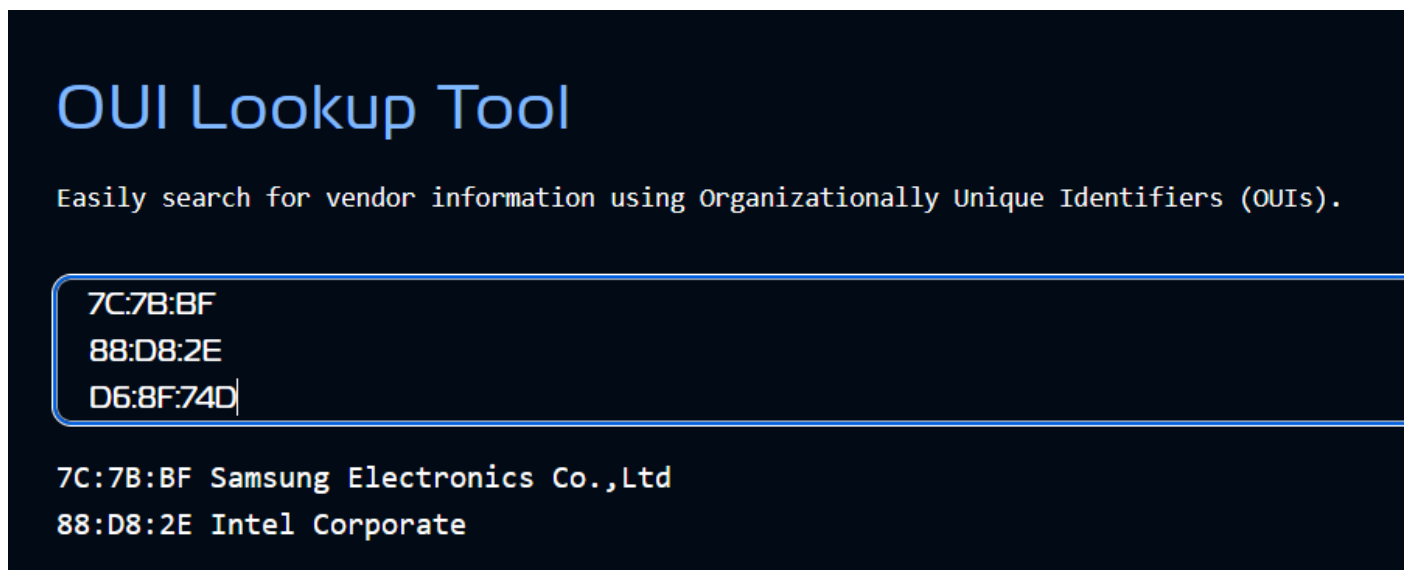


Figure 1.3 — QUI Lookup tool

4. Describe shortly what are these network devices, functions, and services:

- **Repeater** — Receives a signal and retransmits it to extend the communication distance.
- **Hub (multiport repeater)** — Connects multiple devices and forwards incoming data to all ports, regardless of the destination.
- **Bridge** — Connects two network segments and forwards traffic only when necessary based on MAC addresses.
- **Access switch** — Connects end devices such as PCs, printers, and access points to the local network.
- **Core switch** — A high-performance switch that interconnects access switches and carries large amounts of network traffic.
- **Edge router** — Connects an organization's network to external networks, such as the Internet.
- **Core router** — Routes traffic between major networks within a service provider or large enterprise backbone.
- **Firewall** — Monitors and filters network traffic according to security rules to block unauthorized access.
- **Wifi AP** — Provides wireless network access and connects Wi-Fi devices to the wired network.
- **WLAN AP controller** — Centrally manages multiple wireless access points, including configuration, security, and roaming.
- **Network TAP** — A monitoring device that copies network traffic for analysis without affecting the original traffic flow.

5. RFC assignments

- What are RFCs?

RFCs (Request for Comments) are technical documents that describe how Internet technologies, protocols, procedures, and standards work. They are published by organizations such as the **Internet Engineering Task Force (IETF)** and serve as the official reference for many Internet standards.

Historically, RFCs started as informal documents shared among researchers working on the early ARPANET. Despite the name, many RFCs today are official Internet standards.

- How many PPP related RFC documents can you find from rfc-editor website?

The screenshot shows the RFC Editor search interface. The search bar contains 'ppp' and shows 126 results. The filters on the left include 'Search only Metadata and Abstract' (unchecked), 'Search also obsoleted / historic' (checked), and a status filter showing counts for various RFC statuses: Internet Standard (2), Best Current Practice (3), Proposed Standard (66), Draft Standard (8), Informational (38), Experimental (3), and Historic (6). The search results list 'RFC 3818: BCP 88: IANA Considerations for the Point-to-Point Protocol (PPP)' by V. Schryver, dated June 2004, with a status of 'BEST CURRENT PRACTICE'. The abstract describes the charter of the PPP Extensions working group and the allocation of PPP protocol numbers.

Figure 1.4 — Search results for `ppp` on <https://www.rfc-editor.org> has 126 results.

- What is the current status of RFC1597? What is the number for updated, more recent RFC of same topic?

The screenshot shows the RFC Editor search interface for 'RFC1597'. It displays 1 result. The filters on the left include 'Search only Metadata and Abstract' (unchecked), 'Search also obsoleted / historic' (checked), and a status filter showing 'Informational' (1). The search results list 'RFC 1597: Address Allocation for Private Internets' by Y. Rekhter, B. Moskowitz, D. Karrenberg, and G. de Groot, dated March 1994, with a status of 'INFORMATIONAL'. It notes that RFC 1597 is 'Obsoleted by RFC 1918'. The abstract describes methods to preserve IP address space by not allocating globally unique IP addresses to hosts private to an enterprise.

Figure 1.5 — RFC 1597 is Obsoleted by RFC 1918

- When was RFC5218 released?

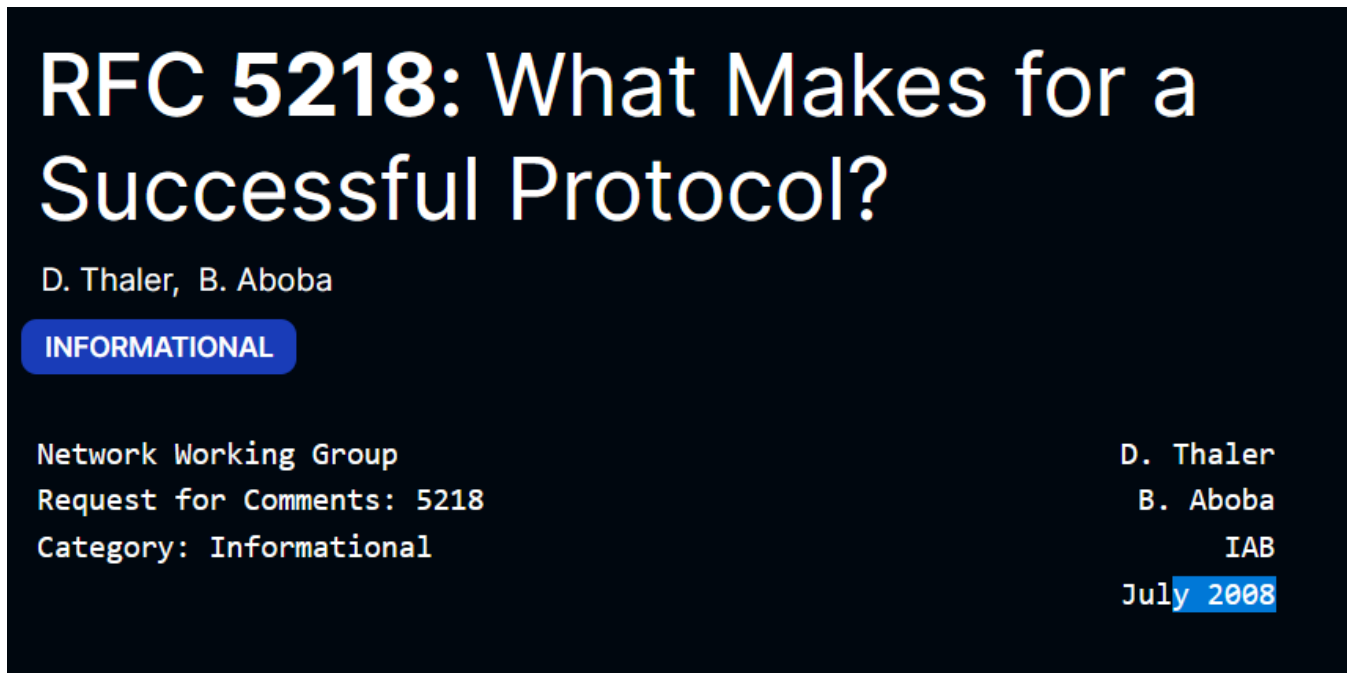


Figure 1.6 — RFC 5218 was released in July 2008

- What is the meaning if RFC status is BCP?

BCP stands for Best Current Practice.

An RFC with status BCP is not necessarily a protocol standard. Instead, it documents the recommended way to do something on the Internet based on operational experience and community consensus.

- List authors of the CoAP RFC (June 2014). What is the RFC number?

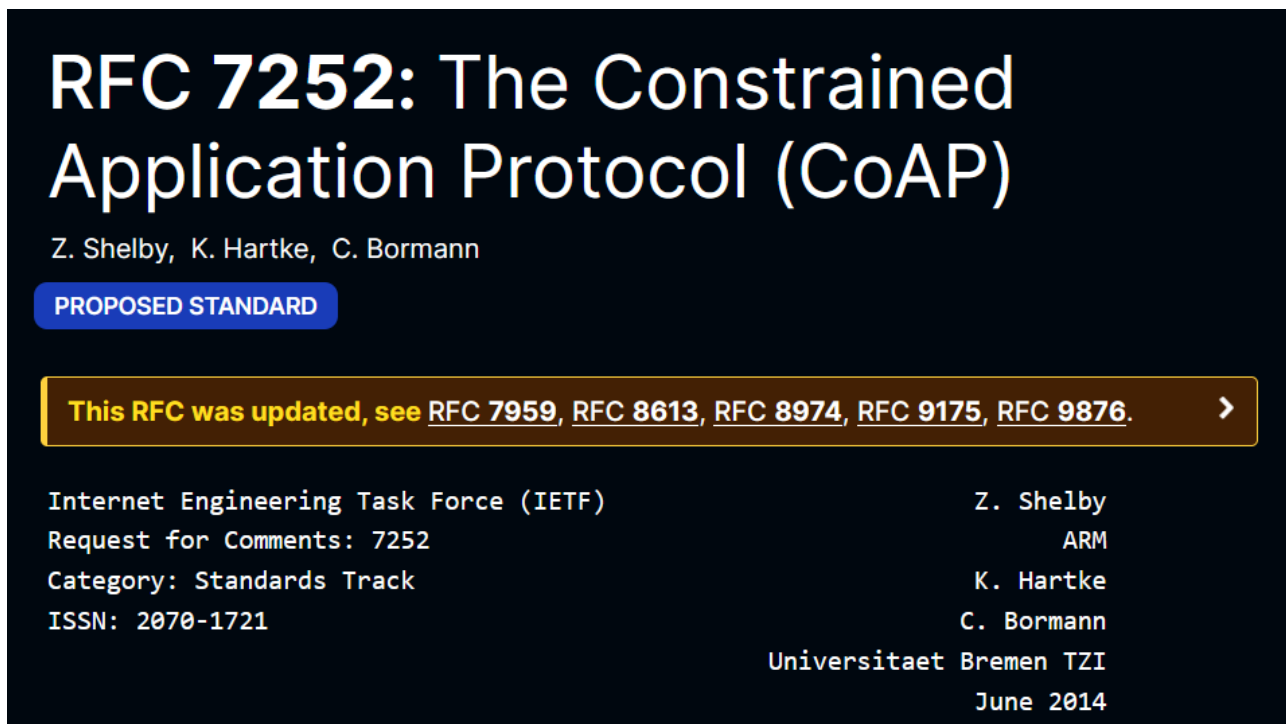


Figure 1.7 — RFC 7252: The Constrained Application Protocol (CoAP)

Authors:

- Z. Shelby
 - K. Hartke
 - C. Bormann
- Twitch.tv provides IRC access to the stream chats. Which RFC defines the original Internet Relay Chat (IRC) Protocol?



Figure 1.8 — RFC 1459. This is the earliest RFC I found on IRC.

6. What is OSI model? Compare OSI model to TCP/IP model

OSI model

- ▶ OSI model (Open System interconnection) by ISO (International Organization for Standardization)
- ▶ Introduced 1984
- ▶ 7 layer model to describe networking process
- ▶ Defines common terminology and helps learning

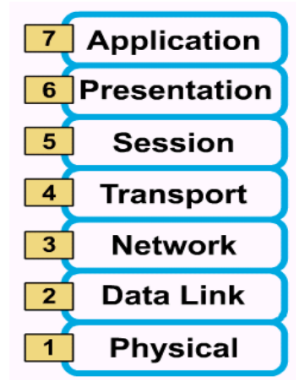


Figure 1.9 — OSI model slide

The *OSI (Open Systems Interconnection)* model is a conceptual framework that describes how data travels through a network. It divides network communication into seven layers, each with specific responsibilities.

TCP/IP model

- ▶ OPS layers 5,6 and 7 combined to Application layer
- ▶ OSI layers 1 and 2 combined to network access layer
- ▶ Sometimes used to describe IP networking

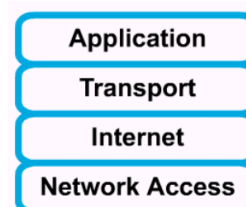


Figure 1.10 — TCP/IP model slide

TCP/IP vs OSI model

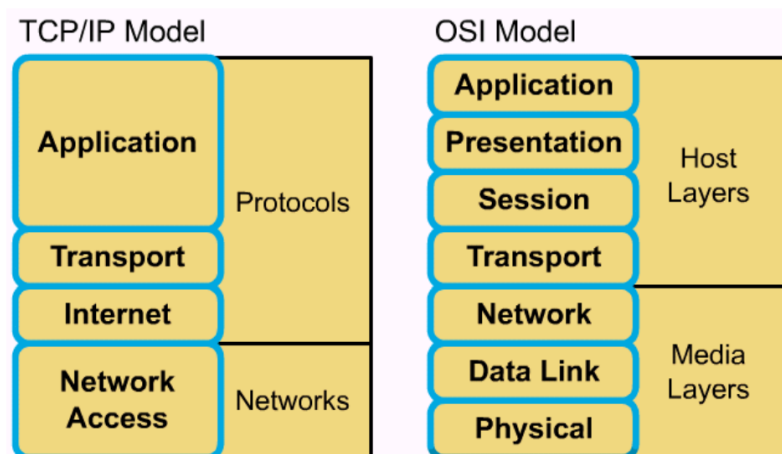


Figure 1.11 — TCP/IP vs OSI model

The **TCP/IP model** describes the same communication with only four layers, and it's the model the real Internet is built on:

OSI model	TCP/IP model	Example protocols / PDU
7 Application, 6 Presentation, 5 Session	Application	HTTP, MQTT, DNS, TLS — data
4 Transport	Transport	TCP (segment), UDP (datagram)
3 Network	Internet	IP, ICMP — packet
2 Data Link, 1 Physical	Network Access (Link)	Ethernet, Wi-Fi — frame, bits

Main differences:

- **Number of layers:** OSI has seven layers. TCP/IP has four, because it merges the top three OSI layers into *Application* and the bottom two into *Network Access*.
- **Purpose:** OSI is a theoretical reference model, designed before the protocols existed. It's mostly used for teaching and troubleshooting ("is it a layer 2 or layer 3 problem?"). TCP/IP was built from protocols that already worked, and it describes how the Internet actually runs.
- **Separation:** OSI keeps presentation (encoding, encryption) and session handling as separate layers. In TCP/IP the application handles them itself, for example TLS inside the application layer.

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

Autumn 2026

Week 2

7. What are VLANs and IEEE 802.1q?

A **VLAN (Virtual LAN)** divides a physical network into separate logical networks, allowing different groups of devices to communicate as if they were on separate switches. Each VLAN typically has its own IP subnet, which improves security, reduces broadcast traffic, and simplifies network management. Devices in different VLANs cannot communicate directly without a router or Layer 3 switch.

IEEE 802.1Q is the standard that adds VLAN tags to Ethernet frames, allowing multiple VLANs to share the same network link between switches.

8. Define following terms and concepts shortly:

- **ARP (Address Resolution Protocol)** maps an IPv4 address to a MAC address on a local network so Ethernet frames can be delivered to the correct device.
- **ARP Spoofing** — a network attack where a device sends fake ARP messages to associate its MAC address with another device's IP address, allowing traffic interception or redirection.
- **Hop (Networking)** — a single step a packet takes through a router on its journey from source to destination. Each router crossed counts as one hop.
- **IP TTL (Time To Live)** is a field in an IP packet that limits how many hops it can traverse. Each router decreases the TTL by 1; when it reaches 0, the packet is discarded.
- **IP TOS (DSCP)** — Type of Service (TOS), now commonly implemented as **DSCP (Differentiated Services Code Point)**, is used to prioritize network traffic such as voice, video, or normal data.
- **DHCP and DHCP Relay**

DHCP (Dynamic Host Configuration Protocol): Automatically assigns IP addresses and other network settings to devices.

DHCP Relay: Forwards DHCP requests between clients and a DHCP server located on a different network or VLAN.

- **WoL (Wake-on-LAN)** — feature that allows a computer to be powered on remotely by sending a specially crafted "magic packet" over the network.
- **UPnP (Universal Plug and Play)** — a protocol that lets applications automatically create port forwarding rules on a router, simplifying connectivity for games and applications.
- **Traceroute / Tracepath** — Network diagnostic tools that show the path packets take to a destination by identifying the routers (hops) along the route.
- **Network Address Translation (NAT)** — a technique where a router translates private IP addresses to one or more public IP addresses, allowing multiple devices to share an Internet connection.
- **Tier 1 and Tier 2 Networks**
 - Tier 1: Large global networks that can reach all other networks without paying for transit.
 - Tier 2: Networks that peer with some networks but also purchase transit from other providers.
- **Tier 3 ISP** — local or regional Internet Service Provider that mainly provides end-user connectivity and purchases Internet transit from higher-tier providers.
- **Autonomous System (AS / ASN)** — a collection of networks under a single administrative organization that presents a common routing policy on the Internet. Each AS is identified by an ASN (Autonomous System Number) and participates in BGP routing.
- **127.0.0.1 Address** — the IPv4 loopback address, representing the local computer itself. Traffic sent to this address never leaves the device.
- **::1 Address** — the IPv6 equivalent of 127.0.0.1, used as the loopback address for the local device.
- **0.0.0.0/0 and ::/0 in Routing Tables**
 - **0.0.0.0/0** — default route for IPv4.
 - **::/0** — default route for IPv6.

Packets destined for unknown networks are sent via the default route.

- **IPv4 Multicast and Experimental Address Ranges**

Multicast: 224.0.0.0/4 (224.0.0.0 - 239.255.255.255), used for one-to-many communication.

Experimental / Reserved: 240.0.0.0/4 (240.0.0.0 - 255.255.255.254), reserved for future or experimental use and generally not routed on the public Internet.

9. Search some information about AS1741

Data sourced from <https://stat.ripe.net>, <https://bgp.he.net/>

- Which organisation or company advertises AS1741 with BGP?
 - **FUNETAS CSC - Tieteen tietotekniikan keskus Oy**
- List some public peering exchange points the AS1741 connects to?
 - FICIX (Helsinki, 193.110.224.14, 2001:7f8:7:b::1741:1)
 - FICIX Espoo (Espoo, 193.110.226.14, 2001:7f8:7:a::1741:1)
 - FICIX Oulu (Oulu, 193.110.225.14, 2001:7f8:7:c::1741:1)
 - TREX (Tampere, 195.140.192.17, 2001:7f8:1d:4::6cd:1)
- To which regional internet registry (RIR) the AS1741 belongs to?
 - **RIPE NCC** (Fig 2.1)
 - Country: **Finland**
- What is the contact email address/phone/web form if you would need to inform some security or abuse issues to the owner of the AS1741?
 - Email: cert@cert.funet.fi
 - <https://www.funet.fi>

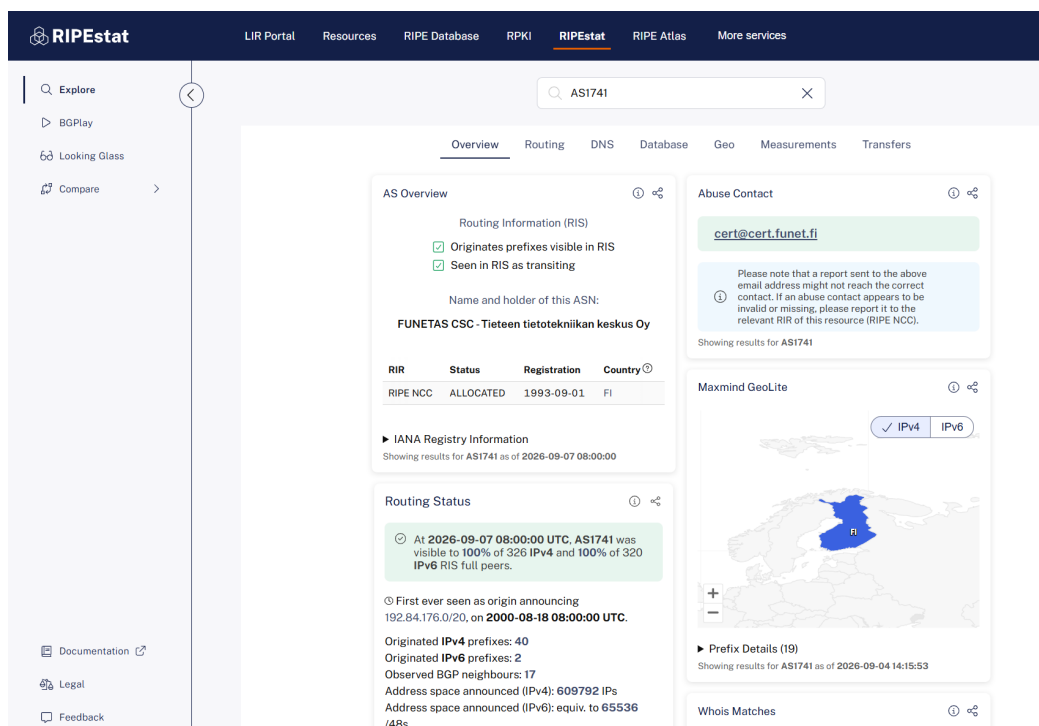


Figure 2.1 — Search result on <https://stat.ripe.net>

AS1741 CSC - Tieteen tietotekniikan keskus Oy.

[illegible]

Figure 2.2 — Search result on <https://bgp.he.net/>

AS1741 CSC - Tieteen tietotekniikan keskus Oy.

Quick Links	AS Info	Graph v4	Graph v6	Prefixes v4	Prefixes v6	Peers v4	Peers v6	Whois	RDAP	IRR	IX	Traceroute
BGP Toolkit Home												
BGP Prefix Report												
BGP Peer Report												
Super Traceroute												
Super Looking Glass												
Cert Search												
Exchange Report												
Bogon Routes												
World Report												
Multi Origin Routes												
DNS Report												
Top Host Report												
RPKI & ASPA Report												
Internet Statistics												
Looking Glass												
Network Tools App												
IPv6 Certification												
IPv6 Progress												
Contribute Data												
Credits												
Contact Us												

Exchange		CC	City	IPv4	
<u>FICIX</u>		FI	Helsinki	193.110.224.14	2
<u>FICIX Espoo</u>		FI	Espoo	193.110.226.14	2
<u>FICIX Oulu</u>		FI	Oulu	193.110.225.14	2
<u>TREX</u>		FI	Tampere	195.140.192.17	2

Figure 2.3 — Exchange points

```

strahus@GllBhh ~$ whois -h whois.ripe.net AS1741
% This is the RIPE Database query service.
% The objects are in RPSL format.
%
% The RIPE Database is subject to Terms and Conditions.
% See https://docs.db.ripe.net/terms-conditions.html
%
% Note: this output has been filtered.
% To receive output for a database update, use the "-B" flag.
% Information related to 'AS1741 - AS1741'
as-block:      AS1741 - AS1741
descr:         RIPE NCC ASN block
remarks:       These AS Numbers are assigned to network operators in the RIPE NCC service region.
mnt-by:        RIPE-NCC-HM-MNT
created:        2018-11-22T15:27:12Z
last-modified: 2018-11-22T15:27:12Z
source:        RIPE
% Information related to 'AS1741'
% Abuse contact for 'AS1741' is 'cert@cert.funet.fi'
aut-num:       AS1741
as-name:       FUNETAS
org:           ORG-FF1-RIPE
import:        from AS565 accept AS565
import:        from AS719 accept AS-ELISAFI
import:        from AS1342 accept AS-ICLFI
import:        from AS1759 accept AS-TSF-Global AS-TSF-National
import:        from AS2603 accept ANY
import:        from AS5400 accept AS-BT-EU
import:        from AS5405 accept AS5405:AS-INTERDOTLINK
import:        from AS6667 accept AS-ELISA

```

Figure 2.4 — `whois -h whois.ripe.net AS1741` command output

10. What is the difference between static and dynamic routing? Use example(s)

Apparently, I misunderstood the question slightly at the beginning. I've kept the section about static and dynamic IP addresses because it is still relevant to the topic, but the first part of the answer has been updated to better address the original question.

Static vs Dynamic Routing

Quite often there is no need for dynamic routing.

Even relatively large organizations may have only a few routers, for example:

- One router connecting all internal subnets
- One firewall connecting the organization to the Internet

In such a network, only a few static routes may be required.

Example

On the router connected to all local networks:

- Configure a default route (0.0.0.0/0) pointing to the firewall.

On the firewall:

- Forward Internet-bound traffic to the Internet Service Provider (ISP).
- Forward traffic destined for internal networks back to the internal router.

In this type of network, static routing is simple, easy to manage, and introduces very little overhead.

When is Dynamic Routing Needed?

Dynamic routing becomes useful when the network topology changes frequently.

Examples include:

- Large enterprise networks
- Networks with multiple interconnected routers
- Networks with redundant links
- Data centers
- Wireless mesh networks
- Large IoT deployments where devices or routing paths may change

Dynamic routing protocols automatically learn routes and can adapt when links or routers fail.

Examples of dynamic routing protocols:

- RIP (Routing Information Protocol)
- OSPF (Open Shortest Path First)
- EIGRP
- IS-IS
- BGP (Border Gateway Protocol)

A major disadvantage of static routing is that routes do not adapt automatically to network failures.

For example:

Router A ---- Router B ---- Router C

If the link between Router B and Router C fails:

- Traffic will continue to be sent toward Router B.
- The packets will not reach their destination.
- Connectivity will be lost until the route is manually changed or a backup route has been configured.

Dynamic routing protocols can automatically detect such failures and select alternative paths.

Routing Metrics

When multiple paths exist, routers use routing metrics to determine the best route.

Common metrics include:

- Bandwidth
- Delay (latency)
- Hop count
- Reliability

- Cost
- Hop Count

Hop count measures the number of routers a packet must pass through.

Example:

Path A: 3 hops

Path B: 5 hops

Using only hop count, Path A would be selected.

However, fewer hops do not necessarily mean better performance. A shorter path may contain a slow or congested link.

Bandwidth

Bandwidth is often a better metric because it reflects the capacity of a link.

Example:

Path A: 3 hops, 10 Mbps

Path B: 5 hops, 1 Gbps

Although Path B has more hops, it may provide significantly better throughput.

RIP and OSPF

RIP (Routing Information Protocol)

- Uses hop count as its metric.
- The route with the fewest hops is preferred.
- Simple to configure.
- Suitable for small networks.
- Maximum path length is 15 hops.

OSPF (Open Shortest Path First)

- Uses a cost metric based primarily on bandwidth.
- Faster convergence than RIP.
- Better suited for medium and large networks.
- Can select higher-bandwidth paths even when they contain more hops.

Distributing IPs locally

▶ Static:

- ▶ Admin configures IP, subnet mask, gw, DNS etc settings manually to hosts
- ▶ Address is fixed and does not change automatically

▶ Dynamic:

- ▶ System requests IP addressing details from network or uses some logic to automatically define IP settings
- ▶ DHCP, IPv6 auto-config

Figure 2.5 — Static vs dynamic slide

With static IP configuration, a network administrator manually sets the network parameters for each device, such as the IP address, subnet mask, default gateway, and DNS servers. On Windows, these settings can be viewed and modified from `Control Panel → Network and Sharing Center → Change adapter settings`.

With dynamic IP configuration, a device automatically receives its network settings from a DHCP server, which is often integrated into a router. This allows network settings to be distributed automatically to many devices, reducing administrative effort. It is common to configure a DHCP reservation, where a specific MAC address is always assigned the same IP address. As a result, even if a device has been offline for some time, it will usually receive the same IP address when it reconnects to the network.

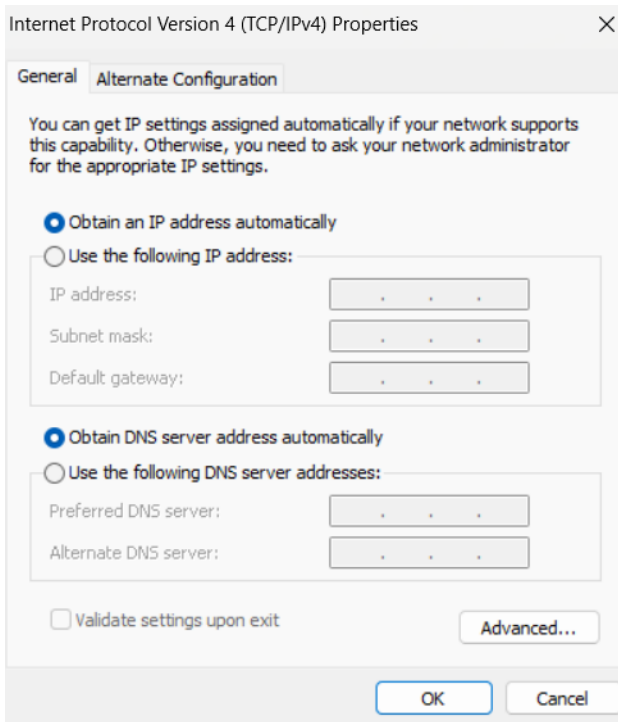


Figure 2.6 — My IPv4 settings

Dynamic Host Configuration Protocol (DHCP)

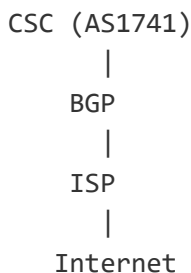
- ▶ DHCP service answers to DHCP requests and supplies IP information
- ▶ Four phases:
 1. DHCP discovery: Client broadcasts request (UDP)
 2. DHCP offer: Server responds and offers free IP from the DHCP pool
 3. DHCP request: Client confirms the offered IP (It is possible to receive several offers)
 4. DHCP acknowledgement: Server sends all IP details + leasing/renewal time

Figure 2.7 — DHCP explained

11. Describe briefly these dynamic routing protocols:

- **RIP (Routing Information Protocol)** is a simple distance-vector routing protocol that selects routes based on the number of hops (routers) to the destination. The maximum path length is 15 hops, which limits RIP to small networks. Routers periodically exchange their routing tables with neighbors.
- **OSPF (Open Shortest Path First) and IS-IS (Intermediate System to Intermediate System)** are link-state routing protocols commonly used in large enterprise and ISP networks.
 - Routers build a map of the network topology.
 - They calculate the shortest path using the Shortest Path First (SPF) algorithm.
 - They adapt quickly to network changes.
 - OSPF is more common in enterprise networks, while IS-IS is often used by large service providers and carrier networks.
- **BGP (Border Gateway Protocol)** is the routing protocol that connects Autonomous Systems (ASes) on the Internet. Unlike RIP or OSPF, BGP focuses on routing policies and AS paths rather than shortest paths. It allows ISPs, universities, and large organizations to exchange routing information globally.

Example:



BGP is often called the routing protocol of the Internet.

- **RPL (Routing Protocol for Low-Power and Lossy Networks)** is designed for IoT (Internet of Things) and sensor networks where devices have limited power, memory, and bandwidth. It organizes devices into a topology called a DODAG (Destination-Oriented Directed Acyclic Graph) and chooses routes optimized for reliability and energy efficiency.

Typical uses:

- Smart homes
- Environmental sensors
- Industrial monitoring
- Wireless sensor networks

12. Create a DNS request (any tool such as ping, nslookup, whatever) to resolve the IP address of www.oamk.fi

To run following commands I used WSL.

```
strahus@G11Bhh ➤ ping -c 1 www.oamk.fi
PING fi-haarla.seravo.com (95.217.107.33) 56(80) bytes of data:
64 bytes from fi-h2.seravo.net (95.217.107.33): icmp_seq=1 ttl=51 time=15.1 ms

--- fi-haarla.seravo.com ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 15.071/15.071/15.071/0.000 ms
strahus@G11Bhh ➤ whois 95.217.107.33
% This is the RIPE Database query service.
% The objects are in RPSL format.
%
% The RIPE Database is subject to Terms and Conditions.
% See https://docs.db.ripe.net/terms-conditions.html
%
% Note: this output has been filtered.
% To receive output for a database update, use the "-B" flag.
%
% Information related to '95.216.0.0 - 95.217.255.255'
% Abuse contact for '95.216.0.0 - 95.217.255.255' is 'abuse@hetzner.com'
inetnum:        95.216.0.0 - 95.217.255.255
netname:        DE-HETZNER-20090224
country:        FI
org:            ORG-HOA1-RIPE
admin-c:        HOAC1-RIPE
tech-c:         HOAC1-RIPE
status:         ALLOCATED PA
mnt-by:         RIPE-NCC-HM-MNT
mnt-by:         HOS-GUN
mnt-lower:      HOS-GUN
mnt-domains:    HOS-GUN
mnt-routes:     HOS-GUN
created:        2009-02-24T07:39:38Z
last-modified:  2017-11-02T11:54:31Z
source:         RIPE # Filtered

organisation:   ORG-HOA1-RIPE
org-name:       Hetzner Online GmbH
country:        DE
reg-nr:         District court Ansbach HRB 6089
org-type:       LIR
address:        Industriestrasse 25
address:        D-91710
address:        Gunzenhausen
address:        GERMANY
phone:          +49 9831 5050
fax-no:         +49 9831 5053
admin-c:        MF1400-RIPE
admin-c:        GM834-RIPE
```

Figure 2.8 — `ping -c 1 www.oamk.fi` and `whois 95.217.107.33` commands output.

From the image we can see that `ping` command gets a reply from a following IP address: 95.217.107.33

- Use some IP whois lookup web service to resolve which company is hosting and has that IP address and server? (www.oamk.fi)
 - Hetzner Online GmbH, DE
- What is the inetnum or route/network (IP address range) the www.oamk.fi's IP address belongs to?
 - 95.216.0.0 - 95.217.255.255
- What is abuse contact email address of that network range?
 - abuse@hetzner.com

13. Use traceroute (tracert in MS Windows command shell) to www.whitehouse.gov

```
C:\git\studies\componentes_IOT_app > main • ?1 ~1 > tracert www.whitehouse.gov

Tracing route to wh47.go-vip.net [192.0.66.51]
over a maximum of 30 hops:

  1      1 ms      1 ms      1 ms  192.168.0.1
  2      1 ms      1 ms      2 ms  78-27-64-2.bb.dnainternet.fi [78.27.64.2]
  3      1 ms      1 ms      2 ms  oul1-sr22.net.dnaip.fi [62.78.100.51]
  4     13 ms     12 ms     15 ms  netnod-ix-ge-a-sth-1500.dna.fi [194.68.123.152]
  5     14 ms     12 ms     12 ms  netnod-ix-ge-a-sth-1500.automattic.com [194.68.123.63]
  6     14 ms     12 ms     12 ms  192.0.66.51

Trace complete.
```

Figure 2.9 — `tracert www.whitehouse.gov` command output

- What is the internet service provider's first router IP address near you? (it's most likely the 2nd router/hop, immediately after your home network)
 - my provider is DNA
 - First router IP: **78.27.64.2**
- How many hops (routers) are there to the www.whitehouse.gov from your device?
 - It took 6 hops from my home network to reach www.whitehouse.gov

- Use traceroute again, but this time to Google's public DNS server in 8.8.8.8, and Quad9 DNS in 9.9.9.9. How far are those?
 - 8.8.8.8 is 8 hops away from my host
 - 9.9.9.9 is 9 hops away

```

C:\git\studies\componentes_IOT_app > main • ?1 ~1 > tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

 1  1 ms    1 ms    1 ms    192.168.0.1
 2  *        *        2 ms    78-27-64-2.bb.dnainternet.fi [78.27.64.2]
 3  2 ms    1 ms    2 ms    oulu-sr22.net.dnaip.fi [62.78.100.51]
 4  *        *        *        Request timed out.
 5  11 ms   12 ms   11 ms   142.250.169.178
 6  11 ms   12 ms   11 ms   142.251.52.197
 7  11 ms   11 ms   11 ms   142.250.229.89
 8  11 ms   16 ms   12 ms   dns.google [8.8.8.8]

Trace complete.

C:\git\studies\componentes_IOT_app > main • ?1 ~1 > tracert 9.9.9.9

Tracing route to dns9.quad9.net [9.9.9.9]
over a maximum of 30 hops:

 1  1 ms    <1 ms   1 ms    192.168.0.1
 2  1 ms    1 ms    2 ms    78-27-64-2.bb.dnainternet.fi [78.27.64.2]
 3  1 ms    1 ms    2 ms    oulu-sr22.net.dnaip.fi [62.78.100.51]
 4  10 ms   10 ms   10 ms   dna.ficix1.ficix.fi [193.110.226.20]
 5  10 ms   10 ms   10 ms   csc.ficix1.ficix.fi [193.110.226.14]
 6  11 ms   10 ms   11 ms   arkisto4.ip.funet.fi [86.50.255.95]
 7  11 ms   10 ms   10 ms   valimo1-et-0-1-0-1.ip.funet.fi [86.50.254.223]
 8  10 ms   10 ms   10 ms   193.167.245.116
 9  10 ms   12 ms   10 ms   dns9.quad9.net [9.9.9.9]

Trace complete.

```

Figure 2.10 — Tracing 8.8.8.8 and 9.9.9.9

- Why traceroute does not always work, and does not show the route up to the final destination IP, or there are timeouts for some routers (* is timeout)? For example, IP address of education.gov.au
 - Traceroute relies on routers sending back ICMP Time Exceeded messages when the packet's TTL (Time To Live) reaches zero. In reality, many routers are configured not to respond to these probes, which causes * timeouts in the output. On WSL for some reason more peers return * * *. Apparently, the mechanism is different (or there is something different in WSL networking settings that I am not aware of)

- Use traceroute and DNS to estimate/guess from response DNS names, round trip times, and with IP whois lookups, where the web server reliefweb.int is located (continent, country or so)?
 - The domain www.reliefweb.int resolves to several AWS IP addresses. A WHOIS lookup of 100.27.176.231 shows that it belongs to the AMAZON-IAD network, operated by Amazon Data Services Northern Virginia. A reverse DNS lookup returns `ec2-100-27-176-231.compute-1.amazonaws.com`, indicating that the server is hosted on Amazon EC2 infrastructure. Based on this evidence, the website is likely hosted in the AWS US East (Northern Virginia, USA) region.

```
strahus@GllBhh ~$ whois www.reliefweb.int
% IANA WHOIS server
% for more information on IANA, visit http://www.iana.org
% This query returned 1 object

domain:      RELIEFWEB.INT
organisation: United Nations OCHA
address:     2 United Nations Plaza
address:     14th Floor
address:     c/o OCHA-IMB-Digital Services
address:     New York New York 10017
address:     USA

contact:     administrative
name:        Saeid Kdaimati
address:     2 United Nations Plaza
address:     14th Floor
address:     c/o OCHA-IMB-Digital Services
address:     New York New York 10017
address:     USA
phone:       +1 917 367 5031
e-mail:      ops@reliefweb.int

contact:     technical
name:        Saeid Kdaimati
address:     2 United Nations Plaza
address:     14th Floor
address:     c/o OCHA-IMB-Digital Services
address:     New York New York 10017
address:     USA
phone:       +1 917 367 5031
e-mail:      ops@reliefweb.int

nservers:    NS-1038.AWSDNS-01.ORG
nservers:    NS-1761.AWSDNS-28.CO.UK
nservers:    NS-51.AWSDNS-06.COM
nservers:    NS-719.AWSDNS-25.NET

created:     1996-07-08
changed:     2021-04-01
```

Figure 2.11 — whois www.reliefweb.int

```

strahus@GllBhh ~ nslookup www.reliefweb.int
Server:      10.255.255.254
Address:     10.255.255.254#53

Non-authoritative answer:
Name:   www.reliefweb.int
Address: 100.51.77.105
Name:   www.reliefweb.int
Address: 52.0.13.60
Name:   www.reliefweb.int
Address: 100.27.176.231
Name:   www.reliefweb.int
Address: 2600:1f18:3379:dd00::f890
Name:   www.reliefweb.int
Address: 2600:1f18:3379:dd02:7ad8:ec0c:3bc6:f2dc
Name:   www.reliefweb.int
Address: 2600:1f18:3379:dd04:52aa:9ba4:192d:28bf

```

Figure 2.12 — nslookup www.reliefweb.int output

14. Use Ficix statistics web page and answer:

- What is the most quiet IP traffic hour in the Ficix 1 exchange point?
 - 04:00-05:00 seems to be the most quiet hour



Figure 2.13 — IP traffic over time

- Which organisations or companies are connected to Ficix 3?
 - CSC
 - Cinia
 - DNA
 - Elisa AS719
 - FNE
 - GleSYS
 - Kaisnet
 - Lounea
 - Telia
 - Valoo

FICIX statistics

• **Total traffic over FICIX**

• **FICIX1**

• **FICIX2**

• **FICIX3**

• **Total traffic**

• **CSC**

• **Cinia**

• **DNA**

• **Elisa AS719**

• **FNE**

• **GleSYS**

• **Kaisanet**

• **Lounea**

• **Telia**

• **Valoo**

Figure 2.14 — FICIX 3: connected companies

15. List all private IPv4 networks (RFC1918)

RFC 1918 defines three private IPv4 address ranges:

CIDR	Address Range	Number of Addresses
10.0.0.0/8	10.0.0.0 - 10.255.255.255	16,777,216
172.16.0.0/12	172.16.0.0 - 172.31.255.255	1,048,576
192.168.0.0/16	192.168.0.0 - 192.168.255.255	65,536

16. What is the purpose of IPv4 private networks?

IPv4 has only about 4.3 billion addresses, which is not enough for every device in the world to have a unique public IP address. Private address ranges allow organizations to reuse the same addresses internally.

For example, millions of networks can simultaneously use **192.168.0.1**, **192.168.0.100** without causing any conflicts. Only a router will need a public IP address. Devices behind the router use private ip addresses and communicate with the Internet through NAT (Network Address Translation).

17. List and explain three or more purposes and features of the ICMP and or ICMPv6 protocol

1. Error Reporting

ICMP informs a sender when a packet cannot be delivered.

Examples:

- Destination Unreachable
- Port Unreachable
- Network Unreachable

2. Network Diagnostics (Ping)

The ping command uses ICMP Echo Request and Echo Reply messages to test connectivity. The sender transmits an Echo Request, and the destination replies with an Echo Reply.

This helps determine:

- Whether a host is reachable.
- Packet loss.
- Round-trip time (RTT).

3. Route Discovery (Traceroute)

`tracert` and `traceroute` rely on ICMP messages to discover the path packets take through a network. When a packet's TTL reaches zero the router sends an ICMP Time Exceeded message. This allows traceroute to identify each hop between the source and destination.

18. Try to solve these basic IP subnet calculations without checking the solutions:

- If network address is 192.168.100.0, and subnet mask is 255.255.255.224, what is the broadcast address of the network?

```
255.255.255.224 = /27
2^(32-27) = 32 addresses
Network: 192.168.100.0
Broadcast: 192.168.100.31
```

- If network address is 1.2.3.4, and broadcast address is 1.2.3.7, what is the subnet mask of the network?

```
Network address 1.2.3.4
and broadcast address 1.2.3.7
will result to 2 usable addresses (4 addresses for the network total):
1.2.3.5
1.2.3.6
```

```
2^(32-prefix) = 4
32 - prefix = 2
prefix = 30
```

```
Subnet mask for /30:
255.255.255.252
```

- If broadcast address is 192.168.129.255 and network mask is 255.255.254.0, what is the network address of the network?

```
255.255.254.0 = /23
```

```
A /23 subnet covers two consecutive /24 networks:
192.168.128.0 - 192.168.129.255
```

```
Network: 192.168.128.0
```

19. Try to solve these IP subnetting assignments without checking the solutions and document at least some examples/answers to the learning diary. Answers should contain (for each subnet): Network address, broadcast address and subnet mask:

Subnetting task 1

The address space available is 172.16.64.0/23. Subnet it and create 5 (A, B, C, D and E) IPv4 subnets with following amount of hosts in each network: A = 85, B = 45, C = 95, D = 57, E = 34.

Leave some small amount of free addresses to each subnet. Avoid unnecessary waste of IPs.

172.16.64.0/23
Subnet mask: 255.255.254.0
Network address: 172.16.64.0
Usable host range: 172.16.64.1 - 172.16.65.254
Broadcast address: 172.16.65.255
Address range: 172.16.64.0 - 172.16.65.255
510 usable addresses

Network A - min 85 hosts
Subnet: 172.16.64.0/25
Network address: 172.16.64.0
Usable host range: 172.16.64.1 - 172.16.64.126
Broadcast address: 172.16.64.127
Address range: 172.16.64.0 - 172.16.64.127
126 usable addresses

Network B - min 45 hosts
Subnet: 172.16.64.128/26
Network address: 172.16.64.128
Usable host range: 172.16.64.129 - 172.16.64.190
Broadcast address: 172.16.64.191
Address range: 172.16.64.128 - 172.16.64.191
62 usable addresses

Network C - min 95 hosts
Subnet: 172.16.65.0/25
Network address: 172.16.65.0
Usable host range: 172.16.65.1 - 172.16.65.126
Broadcast address: 172.16.65.127
Address range: 172.16.65.0 - 172.16.65.127
126 usable addresses

Network D - min 57 hosts
Subnet: 172.16.64.192/26
Network address: 172.16.64.192
Usable host range: 172.16.64.193 - 172.16.64.254
Broadcast address: 172.16.64.255
Address range: 172.16.64.192 - 172.16.64.255
62 usable addresses

Network E - min 34 hosts
Subnet: 172.16.65.128/26
Network address: 172.16.65.128
Usable host range: 172.16.65.129 - 172.16.65.190
Broadcast address: 172.16.65.191
Address range: 172.16.65.128 - 172.16.65.191
62 usable addresses

Subnetting task 2

Same as task 1, but available address space is now 192.168.0.0/25 and networks/hosts are: A = 28, B = 10, C = 60, D = 4.

Leave some small amount of free addresses to each subnet. Avoid unnecessary waste of IPs.

192.168.0.0/25
Subnet mask: 255.255.255.128
Network address: 192.168.0.0
Usable host range: 192.168.0.1 - 192.168.0.126
Broadcast address: 192.168.0.127
Address range: 192.168.0.0 - 192.168.0.127
126 usable addresses

Largest networks first, so each subnet starts on its own boundary:

Network C - min 60 hosts:
Subnet: 192.168.0.0/26
Subnet mask: 255.255.255.192
Network address: 192.168.0.0
Usable host range: 192.168.0.1 - 192.168.0.62
Broadcast address: 192.168.0.63
62 usable addresses

Network A - min 28 hosts:
Subnet: 192.168.0.64/27
Subnet mask: 255.255.255.224
Network address: 192.168.0.64
Usable host range: 192.168.0.65 - 192.168.0.94
Broadcast address: 192.168.0.95
30 usable addresses

Network B - min 10 hosts:
Subnet: 192.168.0.96/28
Subnet mask: 255.255.255.240
Network address: 192.168.0.96
Usable host range: 192.168.0.97 - 192.168.0.110
Broadcast address: 192.168.0.111
14 usable addresses

Network D - min 4 hosts:
Subnet: 192.168.0.112/29
Subnet mask: 255.255.255.248
Network address: 192.168.0.112
Usable host range: 192.168.0.113 - 192.168.0.118
Broadcast address: 192.168.0.119
6 usable addresses

Free: 192.168.0.120 - 192.168.0.127 (8 addresses left for later use).

Subnetting task 3

IPv6 address space available: 2001:708:510::/48. Create four /64 IPv6 networks.

The first 48 bits are fixed: 2001:0708:0510

To create 4 /64 networks we can use next 16 bits as the subnet IDs:

Network A: 2001:708:510:0::/64

Network B: 2001:708:510:1::/64

Network C: 2001:708:510:2::/64

Network D: 2001:708:510:3::/64

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

Autumn 2026

Week 3

20. Answer these questions:

- Explain shortly the purpose of TCP acknowledgment and sequence numbers

Sequence numbers let the receiver reassemble the byte stream in the correct order and detect missing or duplicate data. **Acknowledgment numbers** tell the sender which bytes have been received successfully, so lost segments can be identified and retransmitted.

- What is the purpose of TCP SYN bit?

The **SYN (Synchronize)** flag is used to initiate a connection and synchronize initial sequence numbers between the two hosts during the three-way handshake.

- What is the purpose of TCP reset bit?

The **RST (Reset)** flag immediately aborts a connection, typically sent when a segment arrives for a connection that doesn't exist or when an error requires the connection to be terminated abruptly.

- When TCP retransmissions occur?

Retransmissions occur when the sender doesn't receive an acknowledgment for a segment within the expected time (retransmission timeout), or when duplicate ACKs indicate a segment was lost.

- What is flow-control? (for IP family protocols such as TCP)

Flow control prevents a sender from overwhelming a receiver by sending data faster than it can process. TCP uses a sliding window mechanism where the receiver advertises how much buffer space it has available.

- Explain TCP connection state LISTENING

LISTENING is the state of a server socket that is waiting for incoming connection requests (SYN segments) on a specific port, but no connection has been established yet.

- **Explain TCP connection state ESTABLISHED**

ESTABLISHED means the three-way handshake has completed successfully and both endpoints can now exchange data over the connection.

- **What is the purpose of TCP or UDP source port?**

The **source port** identifies the sending application/process, allowing return traffic to be delivered back to the correct application on the originating host.

- **What is the purpose of TCP or UDP destination port?**

The **destination port** identifies which application or service on the receiving host should process the incoming data.

- **What are the common well-known network service names for these TCP ports: 22, 23, 25, 80, 443, 3306?**

- 22 — SSH (Secure Shell)
- 23 — Telnet
- 25 — SMTP (Simple Mail Transfer Protocol)
- 80 — HTTP
- 443 — HTTPS
- 3306 — MySQL

- **What are common connection-oriented protocol features/advantages, and why TCP is such protocol?**

Connection-oriented protocols establish a session before data transfer, guarantee ordered and reliable delivery, and provide error checking, retransmission, and flow/congestion control. TCP is connection-oriented because it uses the three-way handshake to establish a session, acknowledgments and sequence numbers to guarantee reliable, in-order delivery, and closes the connection explicitly when finished.

- **What are connectionless protocols features (or lack of), and why UDP is connectionless protocol?**

Connectionless protocols send data without establishing a session, provide no guarantee of delivery, ordering, or duplicate protection, and have minimal overhead. UDP is connectionless because it simply sends datagrams to a destination port without a handshake, acknowledgments, or retransmission.

- **Why most services using UDP prefer max 512 byte UDP datagrams?**

512 bytes is small enough to avoid IP fragmentation on virtually all network paths (historically the safe minimum MTU-related size), reducing the risk of packet loss and simplifying delivery since UDP has no built-in retransmission for lost fragments.

- **When it is more reasonable to use UDP instead of TCP?**

UDP is preferable when low latency matters more than reliability, such as for DNS queries, VoIP, video streaming, or gaming, where retransmission delays or connection overhead would be more harmful than an occasional lost packet.

- What is the length of TCP header without extra options? What about UDP header?

The minimum *TCP header* length is 20 bytes. The *UDP header* is fixed at 8 bytes.

- What is TCP Nagle's algorithm? When it should be disabled for networking applications?

Nagle's algorithm buffers small outgoing segments and combines them into fewer, larger packets to reduce network overhead, sending only when a full-sized segment is ready or an ACK is received. It should be disabled (e.g., using the `TCP_NODELAY` socket option) for latency-sensitive applications, such as real-time interactive applications, where small messages need to be sent immediately without buffering delay.

- What is Maximum Transmission Unit (MTU) and IPv4 fragmentation?

The *MTU* is the largest size (in bytes) a single network frame/packet can have on a given link (commonly 1500 bytes for Ethernet). When an IPv4 packet is larger than the MTU of a link it must cross, it is split into smaller pieces — *fragmentation* — which are reassembled at the destination.

- What is a raw socket?

A *raw socket* allows an application to send and receive packets directly at the IP layer (or lower), bypassing the normal TCP/UDP transport processing. It gives direct access to protocol headers, which is useful for tools like ping, traceroute, or custom protocol implementations, and typically requires elevated privileges.

- What is port forwarding?

Port forwarding is a NAT/router configuration that redirects incoming traffic on a specific external port to a specific internal IP address and port, allowing devices behind a NAT/firewall to be reached from the outside network.

21. Describe these protocols or services shortly:

- **IPSec** — a suite of protocols that authenticates and encrypts IP packets, commonly used to build secure VPN tunnels between hosts or networks.
- **RTP and RTCP** — **RTP (Real-time Transport Protocol)** carries real-time audio/video data (e.g., VoIP, video calls) over UDP with timestamps and sequence numbers for playback synchronization. **RTCP (RTP Control Protocol)** runs alongside RTP to provide quality-of-service feedback, such as packet loss and jitter statistics.
- **QUIC (IETF)** — a transport protocol built on top of UDP that provides reliable, ordered, multiplexed streams with built-in TLS encryption and faster connection establishment than TCP+TLS. It is the basis for HTTP/3.
- **Wireguard** — a modern, lightweight VPN protocol that uses state-of-the-art cryptography to create secure, encrypted point-to-point tunnels, known for its simplicity and high performance compared to IPSec/OpenVPN.
- **DoH** — **DNS over HTTPS**, which encrypts DNS queries and responses inside HTTPS traffic, preventing eavesdropping or tampering with DNS lookups.
- **Round-robin DNS** — a load-balancing technique where a DNS name resolves to multiple IP addresses, and the DNS server returns them in rotating order so client requests are distributed across several servers.
- **LDAP** — **Lightweight Directory Access Protocol**, used to query and manage directory services (e.g., user accounts, groups) such as Active Directory over a network.
- **Radius** — **Remote Authentication Dial-In User Service**, a protocol for centralized authentication, authorization, and accounting (AAA), commonly used for network access such as VPNs, Wi-Fi, and switches.
- **Syslog** — a standard protocol/format for sending log messages from devices and applications to a centralized logging server for storage and analysis.
- **NTP** — **Network Time Protocol**, used to synchronize clocks between computers over a network to a common, accurate time source.
- **SNMP** — **Simple Network Management Protocol**, used to monitor and manage network devices (routers, switches, servers) by collecting status information and sending configuration changes.
- **SMTP** — **Simple Mail Transfer Protocol**, used to send and relay email messages between mail servers and from clients to servers.
- **SMB/CIFS** — **Server Message Block / Common Internet File System**, a protocol used mainly on Windows networks for sharing files, printers, and other resources between computers.

22. When listing services with netstat command, what is the meaning if some network service is LISTENING and binded to the IP address 127.0.0.1? What if the service is LISTENING IP address 0.0.0.0?

A service **LISTENING on 127.0.0.1** is bound only to the loopback interface, so it only accepts connections originating from the same local machine — it is not reachable from other hosts on the network.

A service **LISTENING on 0.0.0.0** is bound to all available network interfaces on the host, meaning it accepts connections coming from any IP address/interface, including the local network or the Internet (depending on firewall rules), not just from the local machine.

23. Why some applications are using or offer "keepalive" mechanism to maintain established connection (for example SSH connections)?

Keepalive periodically sends small probe packets over an idle connection to detect whether the other end (or the network path) is still reachable. It prevents intermediate devices, such as NAT routers or firewalls, from silently closing idle connections after a timeout, and allows the application to detect a dead peer quickly and close/reconnect instead of waiting indefinitely on a broken connection.

24. Study available options with command line command "netstat /?" (Windows) or netstat -help (Linux, maybe MacOS). What different things you can check with netstat command?

And here I learned (I've noticed it before but never really asked the question) PowerShell cmdlets use - parameter syntax (for example, `Get-Process -Name notepad`), while external programs such as `netstat`, `ipconfig`, and `net use` use whatever argument style their developers chose, often `/option` or sometimes `-option`. A quick check is `Get-Command <name>`: if it's a Cmdlet, use `Get-Help`; if it's an Application (.exe), use the program's own help such as `/?` or `--help`.

On Windows, `netstat /?` lists the following options:

- `-a` — displays all active connections and listening ports.
- `-b` — displays the executable involved in creating each connection or listening port.
- `-e` — displays Ethernet (interface) statistics, can be combined with `-s`.
- `-f` — displays fully qualified domain names (FQDN) for foreign addresses.
- `-i` — displays the time spent by a TCP connection in its current state.
- `-n` — displays addresses and port numbers numerically, without resolving names.
- `-o` — displays the owning process ID (PID) associated with each connection.
- `-p proto` — filters output to show only connections/statistics for a given protocol (TCP, UDP, TCPv6, UDPv6, IP, IPv6, ICMP, ICMPv6).
- `-q` — displays all connections, listening ports, and bound non-listening TCP ports.
- `-r` — displays the routing table.
- `-s` — displays per-protocol statistics (IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP, UDPv6).
- `-t` — displays the current connection offload state.
- `-x` — displays NetworkDirect connections, listeners, and shared endpoints.

- **-y** — displays the TCP connection template for all connections.
- **interval** — redisplay statistics repeatedly, pausing the given number of seconds between refreshes.

Overall, **netstat** can be used to check active TCP/UDP connections and their states, which ports/services are listening, which process/executable owns each connection, the local routing table, per-protocol/interface traffic statistics, and to continuously monitor connections in near real time.

```
C:\git\studies\componentes_IOT_app > netstat /?
Displays protocol statistics and current TCP/IP network connections.

NETSTAT [-a] [-b] [-e] [-f] [-i] [-n] [-o] [-p proto] [-r] [-s] [-t] [-x] [-y] [interval]

-a          Displays all connections and listening ports.
-b          Displays the executable involved in creating each connection or
           listening port. In some cases well-known executables host
           multiple independent components, and in these cases the
           sequence of components involved in creating the connection
           or listening port is displayed. In this case the executable
           name is in [] at the bottom, on top is the component it called,
           and so forth until TCP/IP was reached. Note that this option
           can be time-consuming and will fail unless you have sufficient
           permissions.
-e          Displays Ethernet statistics. This may be combined with the -s
           option.
-f          Displays Fully Qualified Domain Names (FQDN) for foreign
           addresses.
-i          Displays the time spent by a TCP connection in its current state.
-n          Displays addresses and port numbers in numerical form.
-o          Displays the owning process ID associated with each connection.
-p proto    Shows connections for the protocol specified by proto; proto
           may be any of: TCP, UDP, TCPv6, or UDPv6. If used with the -s
           option to display per-protocol statistics, proto may be any of:
           IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP, or UDPv6.
-q          Displays all connections, listening ports, and bound
           nonlistening TCP ports. Bound nonlistening ports may or may not
           be associated with an active connection.
-r          Displays the routing table.
-s          Displays per-protocol statistics. By default, statistics are
           shown for IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP, and UDPv6;
           the -p option may be used to specify a subset of the default.
-t          Displays the current connection offload state.
-x          Displays NetworkDirect connections, listeners, and shared
           endpoints.
-y          Displays the TCP connection template for all connections.
           Cannot be combined with the other options.
interval    Redisplay selected statistics, pausing interval seconds
           between each display. Press CTRL+C to stop redisplaying
           statistics. If omitted, netstat will print the current
           configuration information once.
```

Figure 3.1 — **netstat /?** command output

```
strahus@GllBhh ~$ netstat -help
usage: netstat [-vWeenNcCF] [<Af>] -r          netstat {-V|--version|-h|--help}
       netstat [-vWnNcaeol] [<Socket> ...]
       netstat [-vWeenNac] -i | [-cnNe] -M | -s [-6tuw] }

-r, --route          display routing table
-i, --interfaces     display interface table
-g, --groups         display multicast group memberships
-s, --statistics     display networking statistics (like SNMP)
-M, --masquerade     display masqueraded connections

-v, --verbose        be verbose
-W, --wide           don't truncate IP addresses
-n, --numeric        don't resolve names
--numeric-hosts      don't resolve host names
--numeric-ports      don't resolve port names
--numeric-users      don't resolve user names
-N, --symbolic       resolve hardware names
-e, --extend         display other/more information
-p, --programs       display PID/Program name for sockets
-o, --timers         display timers
-c, --continuous     continuous listing

-l, --listening      display listening server sockets
-a, --all            display all sockets (default: connected)
-F, --fib            display Forwarding Information Base (default)
-C, --cache          display routing cache instead of FIB
-Z, --context        display SELinux security context for sockets

<Socket>={-t|--tcp} {-u|--udp} {-U|--udplite} {-S|--sctp} {-w|--raw}
          {-x|--unix} --ax25 --ipx --netrom
<Af>=Use '-6' '-4' or '-A <af>' or '--<af>'; default: inet
List of possible address families (which support routing):
inet (DARPA Internet) inet6 (IPv6) ax25 (AMPR AX.25)
netrom (AMPR NET/ROM) ipx (Novell IPX) ddp (Appletalk DDP)
x25 (CCITT X.25)
```

Figure 3.2 — **netstat -help** command output (Ubuntu)

25. Do the 50 ms mystery quiz from <https://mysteries.wizardzines.com/>. What was the cause of extra 50 ms delay?

Here is what was happening:

1. The client opens the connection.
2. The client sends some data (the start of a POST request)
3. The server waits to ACK (because it's using delayed ACKs)
4. The client waits to send more packets (because it's using Nagle's algorithm)
5. Oh no! The server is waiting for the client and client is waiting for the server! We're stuck!!!!
6. After 40ms (that number we've been seeing everywhere!!), the server finally gives up and sends an ACK packet
7. The client sends more data and everything continues

Fix:

Disable Nagle's algorithm on the client side

or

Do not split POST request into 2 parts

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

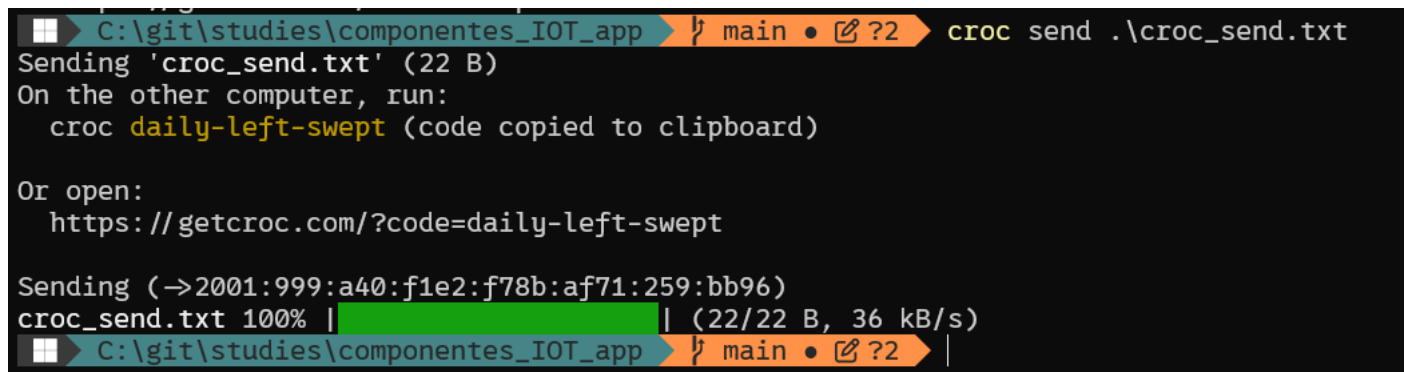
Autumn 2026

Week 4

26. Use **Croc** to move file or files between two or more hosts/devices. Answer shortly:

I have installed **croc** on Windows with `choco install croc` command (PowerShell as Admin).

I have installed **croc** on Android with `pkg install croc` (I use **termux** app).



```
C:\git\studies\componentes_IOT_app > main • ?2 > croc send .\croc_send.txt
Sending 'croc_send.txt' (22 B)
On the other computer, run:
  croc daily-left-swept (code copied to clipboard)

Or open:
  https://getcroc.com/?code=daily-left-swept

Sending (->2001:999:a40:f1e2:f78b:af71:259:bb96)
croc_send.txt 100% | ██████████ | (22/22 B, 36 kB/s)
C:\git\studies\componentes_IOT_app > main • ?2 > |
```

Figure 4.1 — Sending files via **croc** from powershell

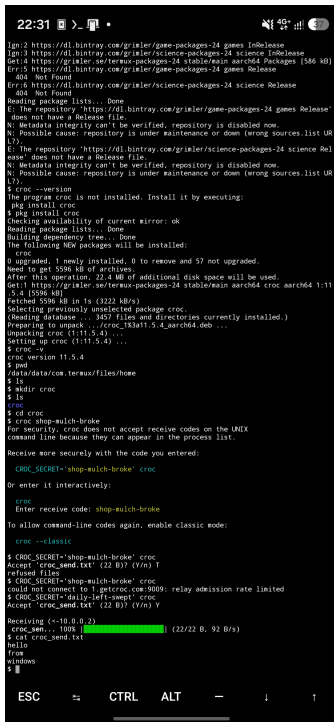


Figure 4.2 — Receive file via `croc` on Android.

I accidentally cancelled the first file send, but on the second try it worked. Also, before the next screenshot I reinstalled the `termux` app. The one I used initially was the outdated version from the Google Playstore. It had quite a few issues. I installed latest version from the F-Droid.

My phone is on cellular data and my windows host is connected to my home Wi-Fi.

Then I have moved the screenshot above from my Android phone to my laptop.

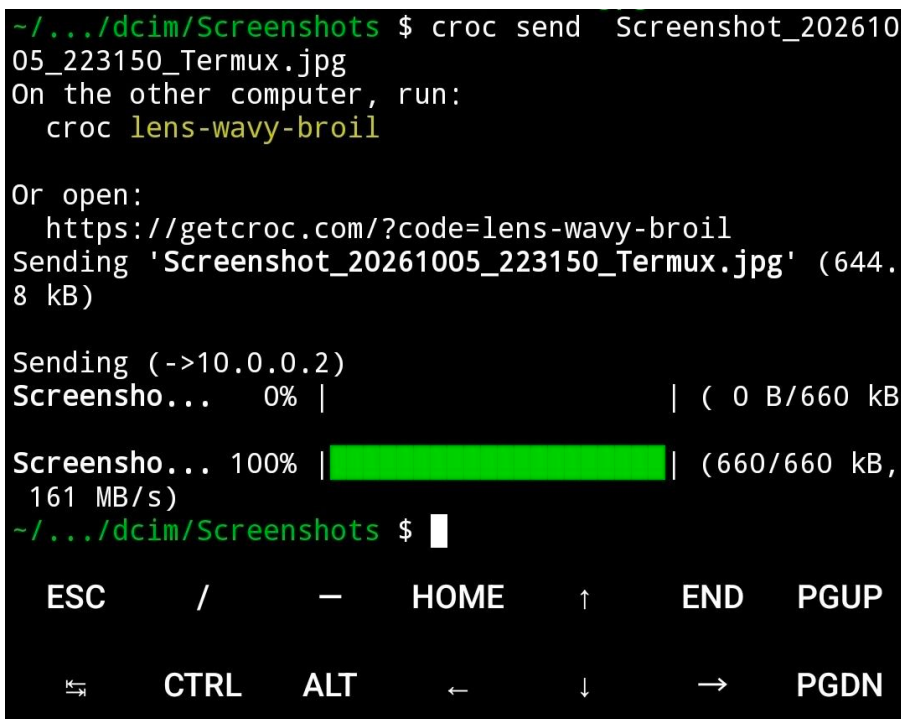
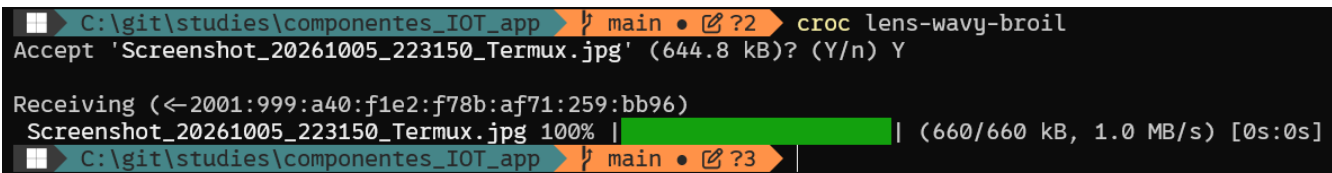


Figure 4.3 — Send a file from Android via `croc`.



```

C:\git\studies\componentes_IOT_app > main • ?2 > croc lens-wavy-broil
Accept 'Screenshot_20261005_223150_Termux.jpg' (644.8 kB)? (Y/n) Y
Receiving (<2001:999:a40:f1e2:f78b:af71:259:bb96)
Screenshot_20261005_223150_Termux.jpg 100% | ██████████ | (660/660 kB, 1.0 MB/s) [0s:0s]
C:\git\studies\componentes_IOT_app > main • ?3 >

```

Figure 4.4 — Screenshot successfully received on Windows.

- **How the Croc works?**

Both sides pick/generate a one-time code phrase. This phrase is used in a **PAKE** (Password Authenticated Key Exchange) handshake, which lets sender and receiver derive the same secret encryption key without ever sending the phrase itself over the network. Croc then encrypts the file (AES-256/ChaCha20) and transfers it directly between the two hosts if they can reach each other, falling back to a relay server otherwise.

- **How the Croc moves files if both hosts are not directly visible to each other? (for example, both are behind NATs or basic firewalls)**

Both hosts only make **outbound** connections to a public (or self-hosted) **relay server** — no incoming ports or port-forwarding are needed. The relay server acts as a rendezvous point: it helps the two peers find each other and, if a direct peer-to-peer connection can't be made (common when both are behind NAT/firewalls), it simply pipes the already end-to-end encrypted traffic between them. Since the data is encrypted before it reaches the relay, the relay operator can't read the file contents.

27. Study how NTP protocol operates and analyse this [Python NTP client code](#). Also available here as [plain text](#).

- This Python script uses direct socket programming to access the NTP server. Comment individual socket programming related code lines. Also, answer these:

- What is the NTP server (DNS) hostname?

`pool.ntp.org` — the default `host` parameter of the `ntp_time()` function.

- What is the destination port number being used?

Port 123, the default `port` parameter of `ntp_time()` and the standard well-known port for the NTP protocol.

- Is this Python script using TCP or UDP? How do you know?

It's using **UDP**. The socket is created with `SOCK_DGRAM` (`socket(AF_INET, SOCK_DGRAM)`), which is the datagram/UDP socket type, as opposed to `SOCK_STREAM` which would indicate TCP. The code also uses `sendto()` / `recvfrom()` instead of `connect()` / `send()` / `recv()`, which is typical of connectionless UDP usage — the socket is never explicitly connected to the server before sending data.

- Try to execute the app with Python

```

C:\git\studies\componentes_IOT_app > python .\week4\src\Python_NTP_client_code.py
Mon Sep 28 21:18:25 2026
C:\git\studies\componentes_IOT_app >

```

Figure 4.5 — Python_NTP_client_code execution in the terminal

28. Do these Python programming assignments with Windows or Linux (or with MacOS if you want and know how)

- For example, use <https://realpython.com/python-sockets/> or similar site(s) for socket programming example codes and create TCP client and TCP server Python scripts
- Establish a TCP connection between your client and server Python scripts (either as localhost traffic or between two separate hosts if you have access to two or more Python running hosts without firewall preventing the traffic)
- Transfer some ASCII text strings between the hosts
 - TCP client connects to the server, sends some plain text string and then disconnects
 - Server prints the text to the console or elsewhere
 - Save your source codes and work. You need scripts again during the course week #5 (Wireshark protocol analyzer assignments)
- Use netstat or similar command line tools to check the TCP connection status (for example the Python server script LISTENING the selected TCP port)

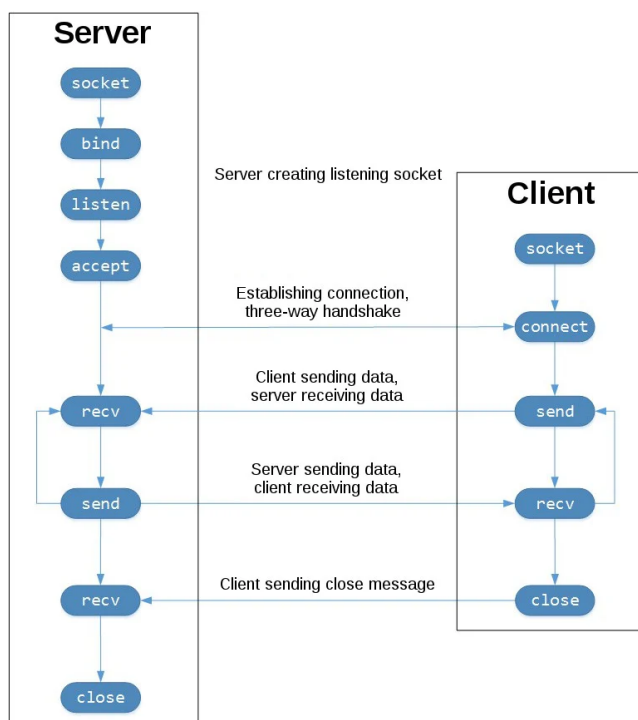


Figure 4.6 — sequence of socket API calls and data flow for TCP (source: <https://realpython.com/python-sockets/>)

I don't have access to another machine now, so my app will run on localhost. I have created two files `tcp_server.py` and `tcp_client.py`.

```
# tcp_server.py
import socket

HOST = "0.0.0.0" # Listen on all interfaces so a remote client can connect too
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Server listening on {HOST}:{PORT}")

    while True:
        conn, addr = server_socket.accept()
        with conn:
            print(f"Connected by {addr}")
            while True:
                data = conn.recv(1024)
                if not data:
                    break
                print(f"Received from {addr}: {data.decode('ascii')}")
            print(f"Disconnected: {addr}")

# tcp_client.py
import socket

HOST = "127.0.0.1" # localhost
PORT = 65432
MESSAGE = "Hello from the TCP client!"

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
    client_socket.connect((HOST, PORT))
    client_socket.sendall(MESSAGE.encode("ascii"))
    print(f"Sent: {MESSAGE}")
```

Figure 4.7 — a line of text sent from the client and received by the server

Figure 4.8 — tcp_server.py app is listening for all incoming connections on port 65432

After I stop the server app the command above returns nothing.

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

Autumn 2026

Week 5

29. Define following terms and concepts shortly:

- What is the difference between encoding and encryption?

Encoding transforms data into another format so it can be stored or transmitted correctly (e.g. binary data as text). The algorithm is public and no key is needed, so anyone can decode it. Its purpose is **usability/compatibility**, not secrecy.

Encryption transforms data so that only someone holding the right **key** can read it. Its purpose is **confidentiality** (and often integrity/authenticity). Without the key, the ciphertext should be practically impossible to reverse.

Base64 looks unreadable, but it is *not* encryption — anyone can decode it.

- List few common encryption algorithms or systems
 - **AES** (Advanced Encryption Standard) — symmetric block cipher, the most widely used today
 - **ChaCha20** (often with Poly1305) — fast symmetric stream cipher, used in TLS and WireGuard
 - **RSA** — asymmetric (public/private key) algorithm, used for key exchange and digital signatures
 - **ECC** (Elliptic Curve Cryptography, e.g. ECDH, ECDSA, Ed25519) — asymmetric, smaller keys than RSA for the same security
 - **3DES / DES** — older symmetric ciphers, now considered obsolete
 - Systems/protocols built on these: **TLS/SSL** (HTTPS), **SSH**, **PGP/GPG**, **IPsec**, **WPA2/WPA3** (Wi-Fi)
- List few common encoding systems
 - **ASCII** and **Unicode** (UTF-8, UTF-16) — character encodings
 - **Base64** — binary data as printable ASCII text (e-mail attachments, data URIs, JWT)
 - **Hexadecimal** — binary bytes as 0-9A-F characters
 - **URL encoding** (percent-encoding) — e.g. space → %20
 - **HTML entities** — e.g. < → <
 - Physical/line encodings like **Manchester** encoding, and media codecs like **MP3/H.264**

- **What are plain text protocols? List some**

Plain text protocols send their commands and data as human-readable, **unencrypted** text over the network. They are easy to debug (you can read them directly in Wireshark), but anyone who can capture the traffic can also read (and modify) it — including usernames and passwords.

Examples: **HTTP, FTP, Telnet, SMTP, POP3, IMAP, DNS** (classic, over UDP/TCP 53), **MQTT** (without TLS), **SNMP v1/v2c**. Most have secured variants: **HTTPS, FTPS/SFTP, SSH, SMTPS, IMAPS, DNS over HTTPS/TLS, MQTTS**.

- **Encapsulation (protocol)**

Encapsulation is how data travels down the network stack: each layer takes the data from the layer above as its **payload** and wraps it with its own **header** (and sometimes trailer). For example:

Application data (HTTP)

- TCP segment [TCP header | HTTP data]
- IP packet [IP header | TCP header | HTTP data]
- Ethernet frame [Eth header | IP header | TCP header | HTTP data | FCS]

The receiver does the reverse (**decapsulation**), stripping one header per layer. This is exactly what Wireshark shows as nested layers when you open a packet.

- **JSON, XML, YAML, CSV**

Common text-based formats for structured data:

- **JSON** (JavaScript Object Notation) — key/value objects and arrays, e.g. `{"temp": 21.5}`. Lightweight and the de facto standard for web APIs and IoT messages.
- **XML** (eXtensible Markup Language) — data in nested tags, e.g. `<temp unit="C">21.5</temp>`. Verbose, but supports attributes, namespaces and schema validation (XSD). Used in SOAP, RSS, office documents and e.g. FMI open data.
- **YAML** (YAML Ain't Markup Language) — indentation-based and very human-readable, e.g. `temp: 21.5`. Mostly used for configuration files (Docker Compose, Kubernetes, GitHub Actions).
- **CSV** (Comma-Separated Values) — flat tabular data, one record per line with fields separated by commas, e.g. `1586544298,21.5`. Simple and compact, ideal for logs and spreadsheets, but has no nesting or data types.

30. Install Wireshark protocol analyser and inspect your IP traffic (DNS requests, web browsing and such) with the Wireshark:

Wireshark is a network packet analyzer. A network packet analyzer presents captured packet data in as much detail as possible.

You could think of a network packet analyzer as a measuring device for examining what's happening inside a network cable, just like an electrician uses a voltmeter for examining what's happening inside an electric cable (but at a higher level, of course).

In the past, such tools were either very expensive, proprietary, or both. However, with the advent of Wireshark, that has changed. Wireshark is available for free, is open source, and is one of the best packet analyzers available today.

from www.wireshark.org

- Analyse the plain text traffic between the TCP socket Python scripts you did during the course week #4.

Note: use localhost network interface when capturing host internal traffic (localhost/127.0.0.1)

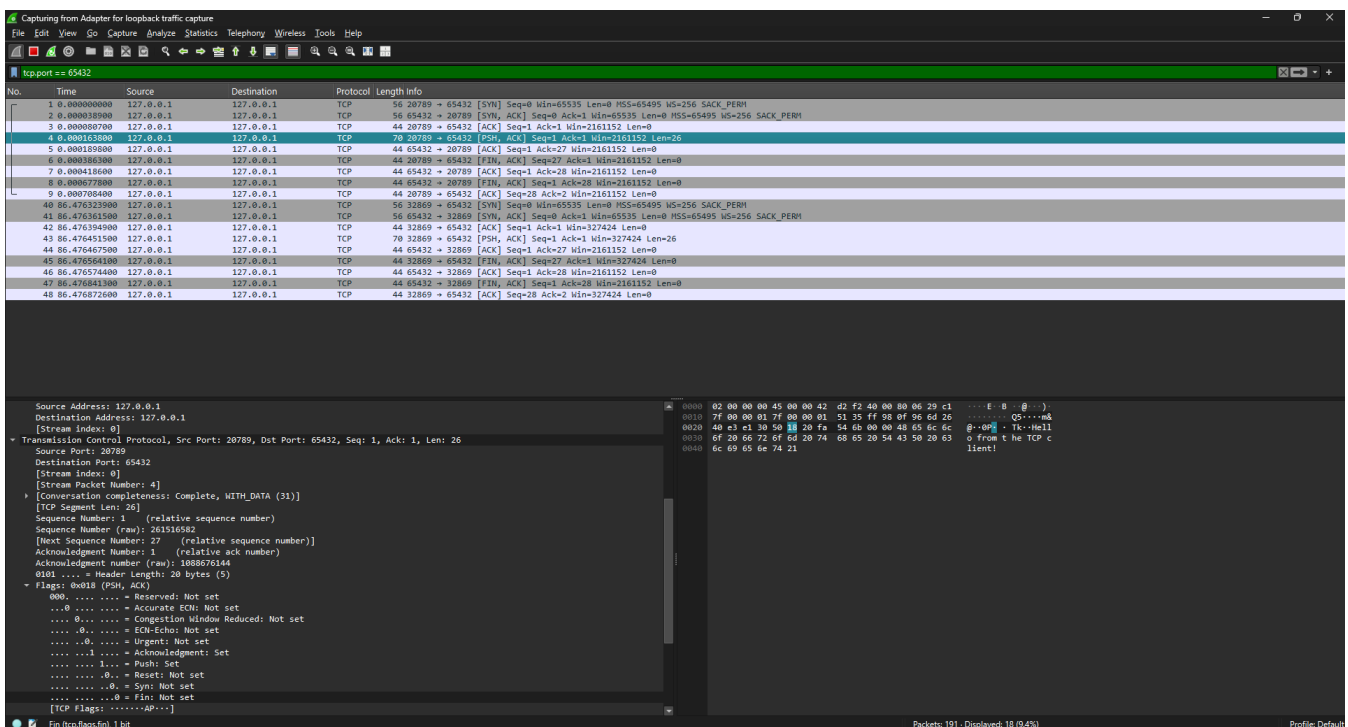


Figure 5.1 — TCP traffic captured with Wireshark. The payload (26 bytes) was transmitted in packet No. 4.

- Try to ping 8.8.8.8 from command prompt and capture the traffic. What protocols ping was using? What is the total header length of your ping request (all used protocol headers combined when ping sends echo request)?

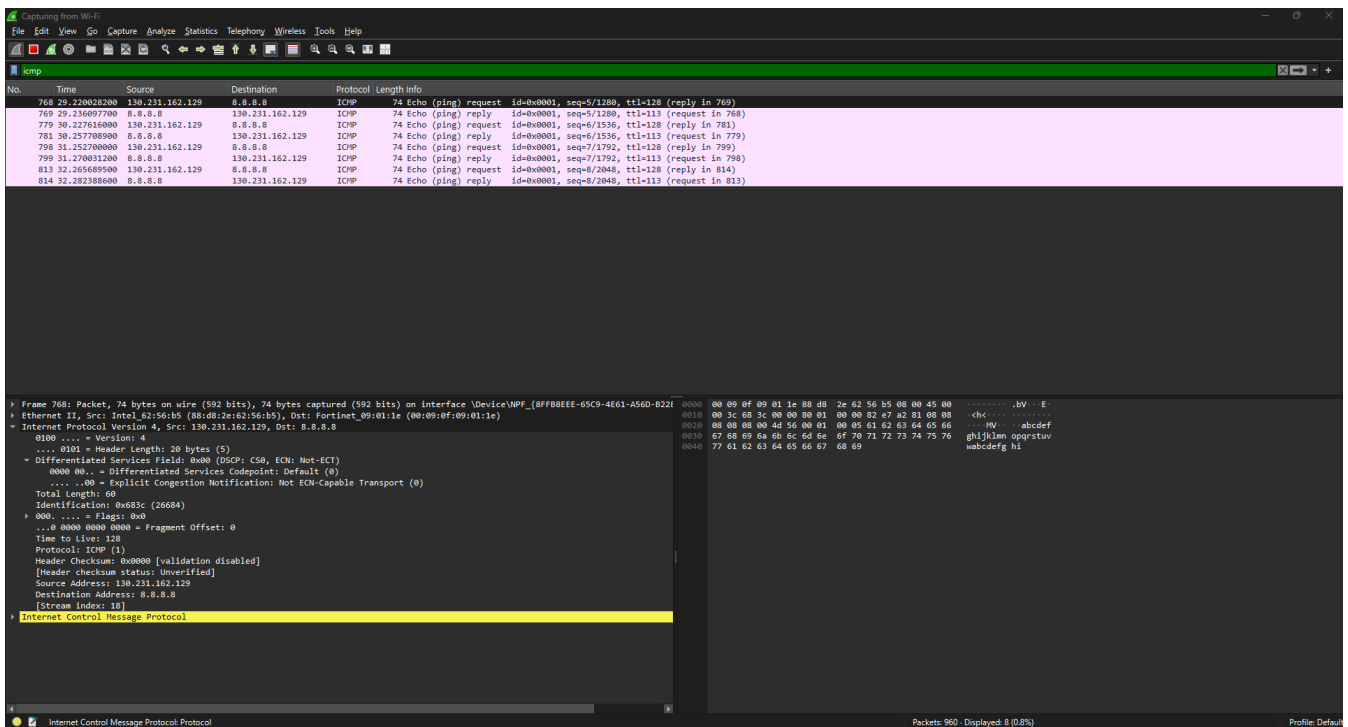


Figure 5.2 — Ping command traffic

Ping uses **ICMP** (Echo request/reply), carried in **IPv4** inside an **Ethernet II** frame. To capture the ping traffic I applied the **icmp** filter.

- Frame Length = 74 bytes
- IP length = 60 bytes
- Therefore the **Ethernet header** is $74 - 60 = 14$ bytes
- The **IP header** is 20 bytes (shown in the **Internet Protocol** details)
- The **ICMP** message is $60 - 20 = 40$ bytes and its data is 32 bytes, so the **ICMP header** is 8 bytes

Header	Size (bytes)
Ethernet	14
IP	20
ICMP	8
Total:	42

- Capture some web browsing traffic and related DNS requests. What are those A (and maybe AAAA requests)? Which protocol is used for DNS requests?

A record - requests the website's IPv4 address.

AAAA record - requests the website's IPv6 address.

Note: This cannot be done with web browser if your browser uses DNS over HTTPS. Most do now. Either skip this task or disable DoH temporarily in the web browser settings.

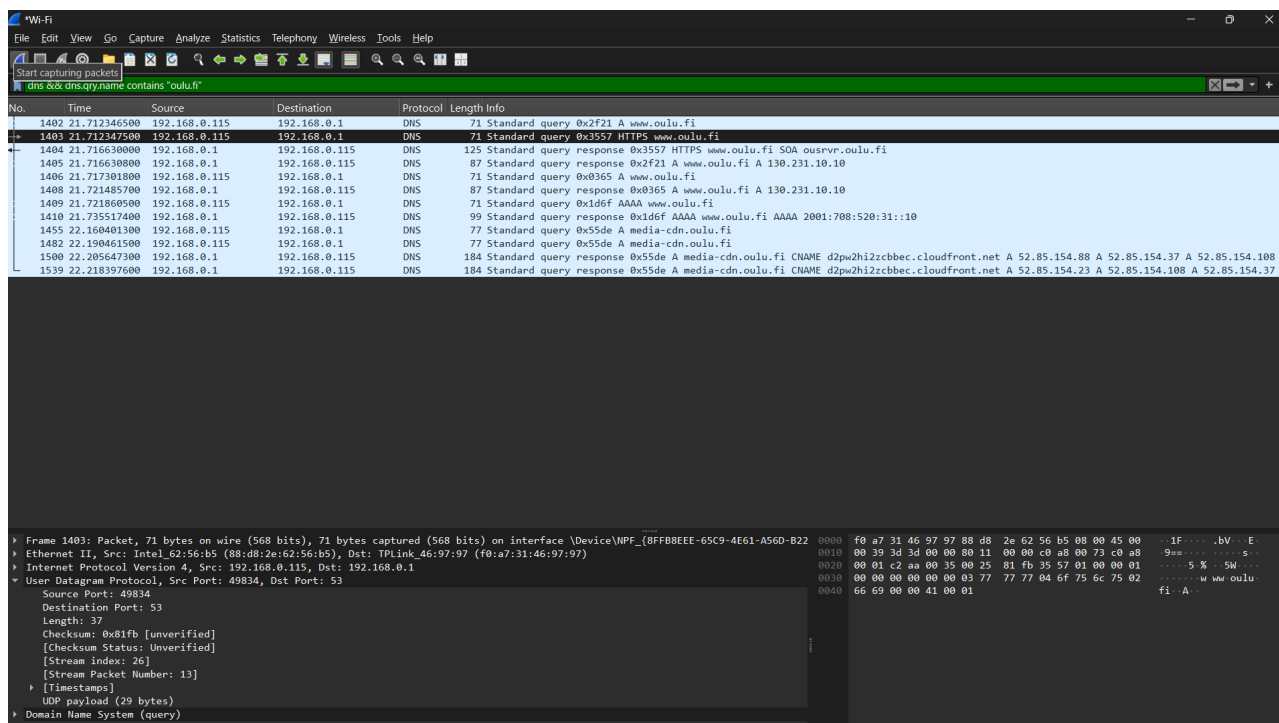


Figure 5.3 — Traffic related to opening www.oulu.fi

To access the website www.oulu.fi, the browser first performed DNS lookups. The capture shows both A and AAAA DNS queries. An A record is used to obtain the IPv4 address of the host, while an AAAA record is used to obtain the IPv6 address. The DNS responses returned IPv4 address 130.231.10.10 and IPv6 address 2001:708:520:31::10 for www.oulu.fi. The DNS requests were transported using the UDP protocol on port 53.

31. Download this [zipper pcap traffic file](#) and inspect it with Wireshark. The IP traffic sample is about IoT device sending base64 encoded and JSON formatted data to a server. Answer these questions:

- What is the total size of captured frame in bits?
 - 1536 bits
- What is the payload length (data) in bytes?
 - 150 bytes
- What is the source IP address of device sending the traffic?
 - 194.163.171.214
- What is the destination IP address receiving the traffic?
 - 193.167.100.28
- What is the IP family protocol delivering the data?
 - IPv4
- What is the source port?
 - 49240
- What is the destination port?
 - 8080

- Extract the payload as printable text (use right mouse button and copy as printable text for the data part only). Use any base64 decoder to convert the data to a plain text JSON message. What is the content of JSON formatted data?

Decode from Base64 format

Simply enter your data then push the decode button.

```
ewogICAgImIucHV0TmFlZSI6ICJPYW1rIElQIG5ldHdvcmtpbmcgY291cnNlIGJ1dHRvbilsCiAgICAgIYnV0dG9uUHJlc3N1cmUiOiAiMTMzNyIsCiAgICAgIidmVyc2lvbi6IClxLjluMyIKKQo=
```

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

AUTO-DETECT Source character set. Detected: UTF-8

☒ Decode each line separately (useful for when you have multiple entries).

☐ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE >

Decodes your data into the area below.

```
{
  "inputName": "Oamk IP networking course button",
  "buttonPressure": "1337",
  "version": "1.2.3"
}
```

Figure 5.4 — Decoded data

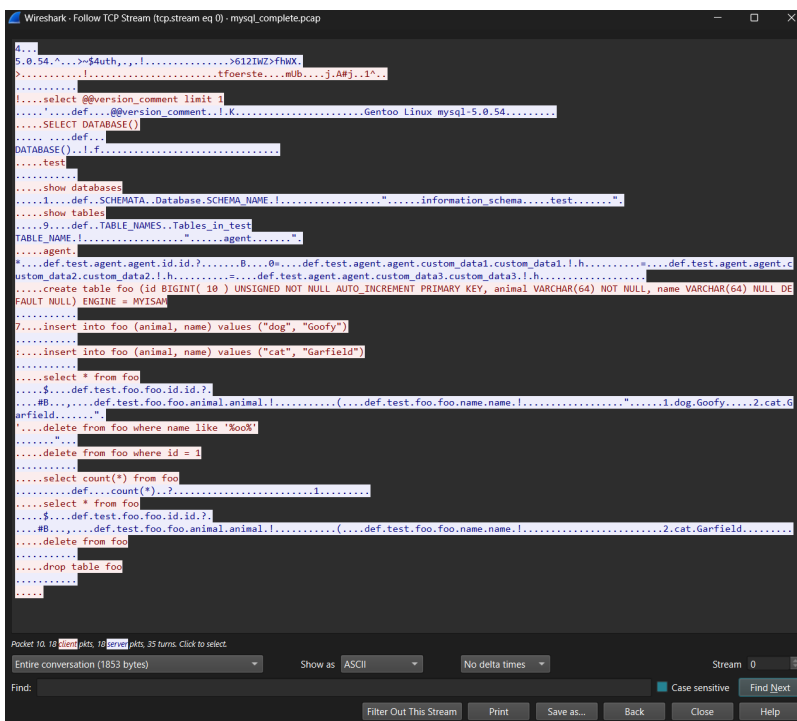
To get the base64 string I right-clicked on the frame and selected `Follow -> UDP Stream`, which allowed me to copy the base64 string. Then I used base64decode.org to decode it:

```
{
  "inputName": "Oamk IP networking course button",
  "buttonPressure": "1337",
  "version": "1.2.3"
}
```

Hint: If you struggle with this task, you can get the UDP payload if you select the UDP datagram with right mouse button in the Wireshark packet view, and use "Follow". It gives you the UDP packet data payload as ASCII text you can copy-paste from the Wireshark popup window to any common base64 decoder.

32. Download this [zipper pcap traffic file](#) and inspect it with Wireshark. Traffic is simple MySQL session example from Wireshark Wiki. Answer these questions:

- What is the destination IP address receiving the traffic?
 - 192.168.0.254
- What is the destination TCP port?
 - 3306
- Use Wireshark's follow TCP stream feature (right mouse button) and inspect what are the two database rows (animals) and related values which were inserted to the foo table's animal and name columns?
 - "dog", "Goofy"
 - "cat", "Garfield"



The screenshot shows the Wireshark interface with the 'Follow TCP Stream' feature enabled for the MySQL session. The stream content is displayed in ASCII format, showing a series of SQL commands. The commands include a version check, a database selection, a table creation for 'foo' with columns 'animal' and 'name', and two insertions: ('dog', 'Goofy') and ('cat', 'Garfield'). The stream also shows a delete operation and a count query. The interface at the bottom indicates the stream is 1853 bytes long and provides options for filtering, printing, and saving the stream data.

Figure 5.5 — MySQL TCP data

33. Download this [zipped pcap traffic file](#) and inspect it with Wireshark. Traffic has been captured from host 192.168.80.32. Answer these questions:

- What is the MAC address of host 192.168.80.32?
 - 08:00:27:f1:90:ad
- What is the MAC address of host 192.168.80.1? Which vendor has build the ethernet chipset of host 192.168.80.1? (use Wireshark or IEEE OUI data)
 - fc:ec:da:4a:84:d3
 - Ubiquiti Inc.
- Which IP address sent ICMP echo requests to this (192.168.80.32) host? Also, there is a repeating short message inside ICMP datagrams the host sent as ICMP echo request payload. What is the repeated message?
 - 192.168.80.58
 - The repeated message is Hi there Oamk! (the payload shows there Oamk!Hi there Oamk!Hi there Oamk!)
- What was the web page the host 192.168.80.32 visited first (full web page address, not just the host)? What was the web browser or HTTP user agent string used to access that web server?
 - <http://www.oamk.fi/~tkorpela/>
 - curl/7.68.0
- What is the hostname in "Host:" field of the HTTP GET request sent by 192.168.80.32?
 - www.oamk.fi
- What is most likely the default DNS server (the IP address) used by the host 192.168.80.32?
 - 8.8.8.8

- Use Wireshark's file/export objects/HTTP feature to extract the ZIP file which was downloaded from the web server 193.167.100.88. What is inside the ZIP file?

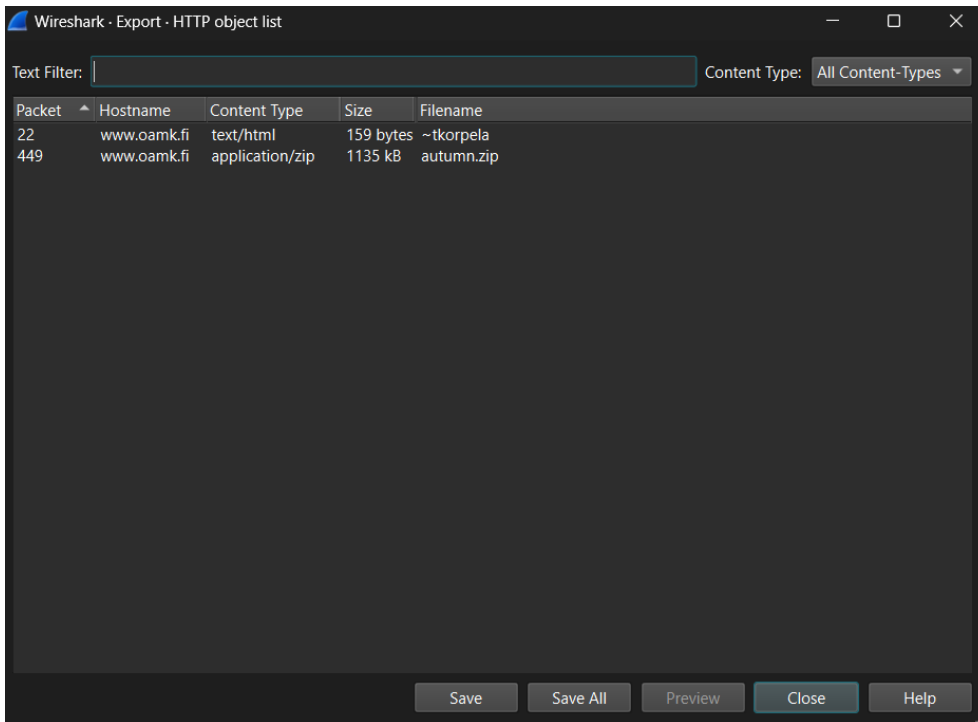


Figure 5.6 — Export shows `~tkorpela` (text/html) and `autumn.zip` (application/zip)

- The ZIP file `autumn.zip` contains a single image, `autumn.jpg`.
- Host 192.168.80.32 sent DNS requests to host 9.9.9.9. What are the requests?

The image shows a Wireshark network capture of a .pcap file. The filter is 'ip.addr == 9.9.9.9'. The packet list shows several DNS queries and responses:

No.	Time	Source	Destination	Protocol	Length	Info
1	*REF*	192.168.80.58	192.168.80.32	ICMP	98	Echo (ping) request id=0x0fb0, seq=1/256, ttl=64 (reply in 2)
454	96.764967	192.168.80.32	9.9.9.9	DNS	79	Standard query 0xf56 A www.teemukorpela.fi
455	96.838558	9.9.9.9	192.168.80.32	DNS	95	Standard query response 0xf56 A www.teemukorpela.fi A 13.48.208.217
460	112.854124	192.168.80.32	9.9.9.9	DNS	69	Standard query 0x1d82 MX nokia.com
461	112.907009	9.9.9.9	192.168.80.32	DNS	119	Standard query response 0x1d82 MX nokia.com MX 10 nokia-com.mail.protection.outlook.com
462	124.638465	192.168.80.32	9.9.9.9	DNS	75	Standard query 0xf9a2 AAAA www.youtube.com
463	124.695746	9.9.9.9	192.168.80.32	DNS	137	Standard query response 0xf9a2 AAAA www.youtube.com CNAME youtube-ui.l.google.com AAAA 2a00:1450:4001:825::20...

Figure 5.7 — Requests to 9.9.9.9

Host 192.168.80.32 sent DNS queries to the public DNS server 9.9.9.9 (Quad9). The queries included an **A** record lookup for `www.teemukorpela.fi`, an **MX** record lookup for `nokia.com` and an **AAAA** record lookup for `www.youtube.com`. The DNS responses returned the corresponding IPv4 address, mail server and IPv6 address. DNS was transported using the UDP protocol.

34. Create a new JSON file with any text editor. JSON file should contain data for at least two houses and related IoT sensor data. Each house must have few sensors with following information and some random data for each sensor. Something like this:

House:

- IoT sensor:
 - sensor ID number
 - location description
 - notes about sensor
 - unix epoch timestamp
 - sensor values:
 - value nnn
 - value nnn
 - value nnn

```

{
  "houses": [
    {
      "houseId": 1,
      "name": "Summer cottage",
      "address": "Rantatie 12, 90100 Oulu",
      "sensors": [
        {
          "sensorId": 1001,
          "type": "temperature_humidity",
          "location": "Living room, north wall, 1.5 m above floor",
          "notes": "Battery powered, sends data every 10 minutes over Zigbee",
          "timestamp": 1791280800,
          "values": {
            "temperature_c": 21.4,
            "humidity_pct": 38.2,
            "battery_pct": 87
          }
        },
        {
          "sensorId": 1002,
          "type": "air_quality",
          "location": "Bedroom, on the bookshelf",
          "notes": "USB powered, CO2 calibrated 2026-09-01",
          "timestamp": 1791280860,
          "values": {
            "co2_ppm": 742,
            "tvoc_ppb": 118,
            "pm2_5_ugm3": 4.6
          }
        },
        {
          "sensorId": 1003,
          "type": "water_leak",
          "location": "Kitchen, under the sink",
          "notes": "Alarm is triggered when leak value is 1",
          "timestamp": 1791280920,
          "values": {
            "leak": 0,
            "temperature_c": 18.9,
            "battery_pct": 64
          }
        }
      ]
    },
    {
      "houseId": 2,
      "name": "Apartment",
      "address": "Kauppurienkatu 5 B 14, 90100 Oulu",
      "sensors": [

```

```

{
  "sensorId": 2001,
  "type": "temperature_humidity",
  "location": "Bathroom, ceiling next to the vent",
  "notes": "High humidity is expected after showers",
  "timestamp": 1791281400,
  "values": {
    "temperature_c": 24.1,
    "humidity_pct": 71.5,
    "battery_pct": 92
  }
},
{
  "sensorId": 2002,
  "type": "energy_meter",
  "location": "Hallway, electrical cabinet",
  "notes": "Measures total power consumption of the apartment",
  "timestamp": 1791281460,
  "values": {
    "power_w": 1245,
    "voltage_v": 231.6,
    "energy_kwh": 5873.42
  }
},
{
  "sensorId": 2003,
  "type": "door_motion",
  "location": "Front door",
  "notes": "Door contact and PIR motion sensor in the same device",
  "timestamp": 1791281520,
  "values": {
    "door_open": false,
    "motion_detected": true,
    "battery_pct": 45
  }
}
]
}

```

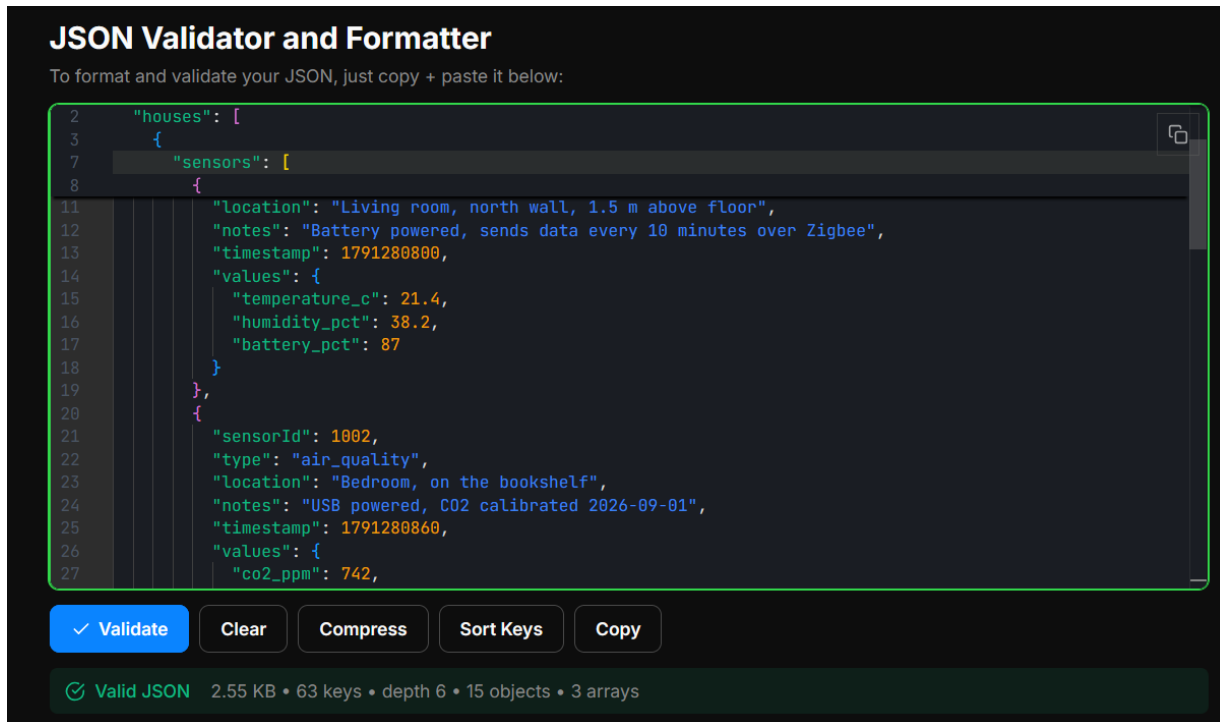


Figure 5.8 — JSON validation

What is GraphQL? Also, check this [traffic and parking API documentation from Oulu](#) (extra task uses this API)

- **GraphQL** is a query language for APIs and a server-side runtime for running those queries. Facebook developed it in 2012 and released it as open source in 2015. A REST API has many endpoints, each returning a fixed data structure. A GraphQL API usually has **one endpoint** (`POST /graphql`), and the client sends a query that says exactly which fields it wants. The response is JSON with the same shape as the query.
 - **Schema & types:** the server describes its data with a strongly typed schema (types, fields, relations). Clients can read this schema through *introspection*, which makes self-documenting tools such as GraphiQL possible.
 - **Operations:** `query` reads data, `mutation` changes data and `subscription` gets real-time updates, usually over WebSockets.
 - **Advantages:** no over-fetching (getting fields you don't need) and no under-fetching (needing several requests). Related data comes back in one round trip, which is useful for IoT and mobile clients with limited bandwidth.
 - **Disadvantages:** HTTP caching is harder because everything goes through one POST endpoint. Very complex or deep queries can overload the server. It is also more complex to set up than a simple REST API.

- Oulu traffic and parking API ([oulunliikenne.fi](https://api.oulunliikenne.fi))

- Endpoint: `https://api.oulunliikenne.fi/proxy/graphql`
- Available queries: `carParks` (parking garages with real-time free spaces), `cameras`, `tmsStations` (traffic volume/speed), `weatherStations`, `roadworks`, `trafficAnnouncements`, `maintenanceVehicleRouteEvents`, `maintenanceVehicleObservations` and `trafficFluencyFeatureCollection`.
- Timestamps are in UTC (ISO 8601) and coordinates use GeoJSON.

Example query: the name and free spaces of every car park:

```
{
  carParks {
    carParkId
    name
    spacesAvailable
    maxCapacity
  }
}
```

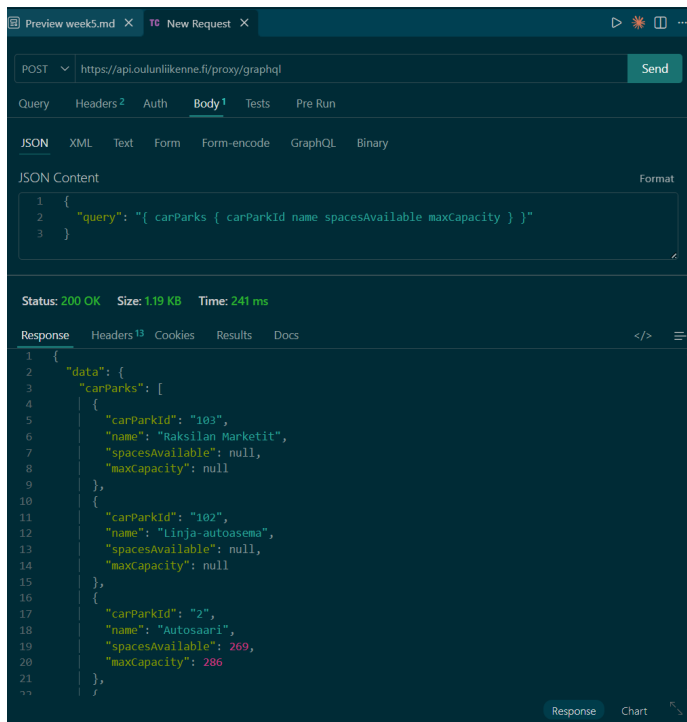
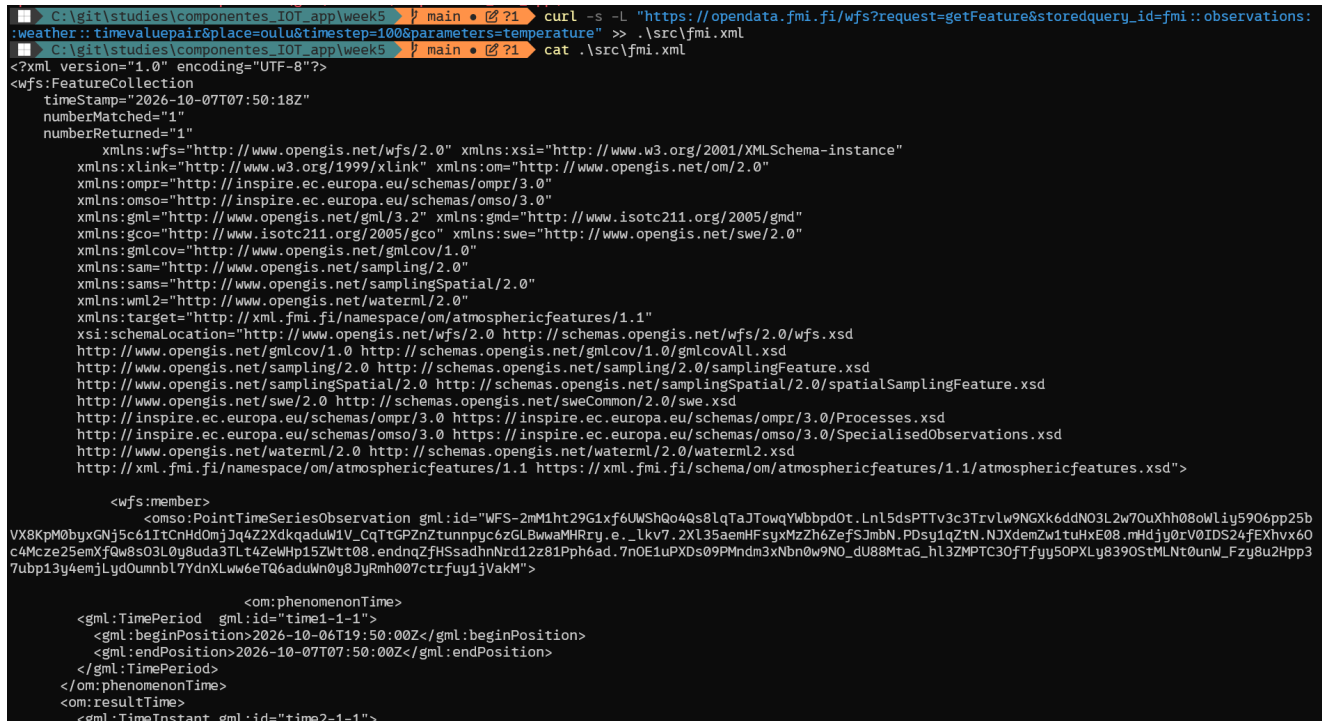


Figure 5.9 — Oulu traffic parking API test with Thunder Client

35. Install Cmdr (or some other toolset where you have Curl or similar tool to make HTTP requests from command line or application.) Use Curl to fetch XML formatted weather data from FMI:

```
curl -s -L "https://opendata.fmi.fi/wfs?request=getFeature&storedquery_id=fmi::observations::weather::timevaluepair&place=oulu&timestep=100&parameters=temperature" >> ./src/fmi.xml
```



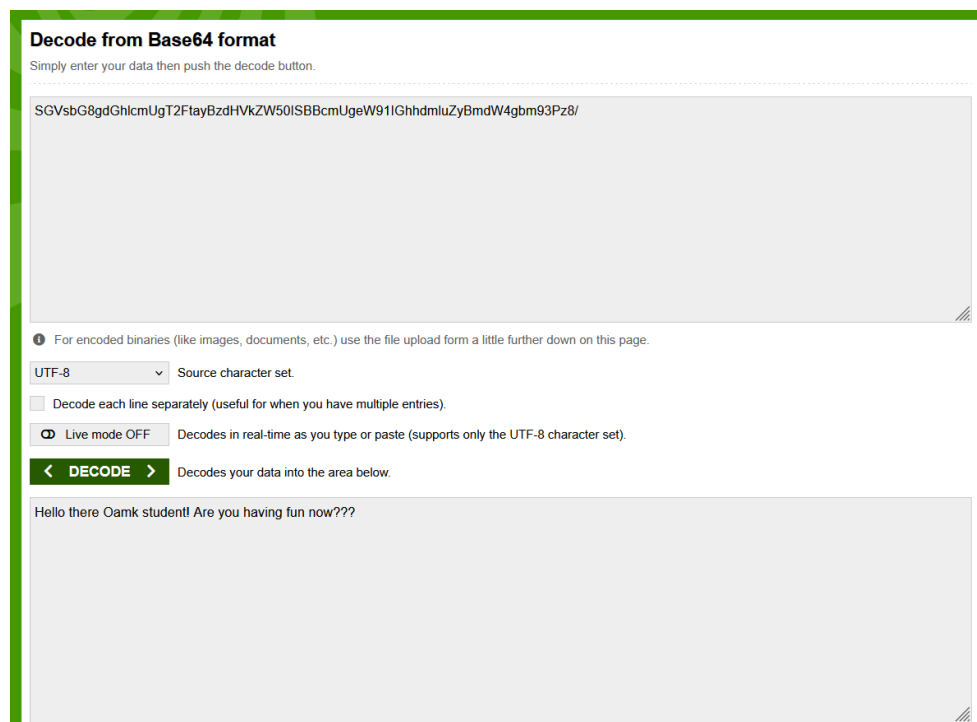
```
C:\git\studies\componentes_IOT_app\week5 > main • ?1 curl -s -L "https://opendata.fmi.fi/wfs?request=getFeature&storedquery_id=fmi::observations::weather::timevaluepair&place=oulu&timestep=100&parameters=temperature" >> ./src/fmi.xml
C:\git\studies\componentes_IOT_app\week5 > main • ?1 cat ./src/fmi.xml
<?xml version="1.0" encoding="UTF-8"?>
<wfs:FeatureCollection
  timeStamp="2026-10-07T07:50:18Z"
  numberMatched="1"
  numberReturned="1"
  xmlns:wfs="http://www.opengis.net/wfs/2.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xlink="http://www.w3.org/1999/xlink" xmlns:om="http://www.opengis.net/om/2.0"
  xmlns:ompr="http://inspire.ec.europa.eu/schemas/ompr/3.0"
  xmlns:omso="http://inspire.ec.europa.eu/schemas/omso/3.0"
  xmlns:gml="http://www.opengis.net/gml/3.2" xmlns:gmd="http://www.isotc211.org/2005/gmd"
  xmlns:gco="http://www.isotc211.org/2005/gco" xmlns:swe="http://www.opengis.net/swe/2.0"
  xmlns:gmlcov="http://www.opengis.net/gmlcov/1.0"
  xmlns:sam="http://www.opengis.net/sampling/2.0"
  xmlns:sams="http://www.opengis.net/samplingSpatial/2.0"
  xmlns:wml2="http://www.opengis.net/waterml/2.0"
  xmlns:target="http://xml.fmi.fi/namespace/om/atmosphericfeatures/1.1"
  xsi:schemaLocation="http://www.opengis.net/wfs/2.0 http://schemas.opengis.net/wfs/2.0/wfs.xsd
    http://www.opengis.net/gmlcov/1.0 http://schemas.opengis.net/gmlcov/1.0/gmlcovAll.xsd
    http://www.opengis.net/sampling/2.0 http://schemas.opengis.net/sampling/2.0/samplingFeature.xsd
    http://www.opengis.net/samplingSpatial/2.0 http://schemas.opengis.net/samplingSpatial/2.0/spatialSamplingFeature.xsd
    http://www.opengis.net/swe/2.0 http://schemas.opengis.net/sweCommon/2.0/swe.xsd
    http://inspire.ec.europa.eu/schemas/ompr/3.0 http://inspire.ec.europa.eu/schemas/ompr/3.0/Processes.xsd
    http://inspire.ec.europa.eu/schemas/omso/3.0 http://inspire.ec.europa.eu/schemas/omso/3.0/SpecialisedObservations.xsd
    http://www.opengis.net/waterml/2.0 http://schemas.opengis.net/waterml/2.0/waterml2.xsd
    http://xml.fmi.fi/namespace/om/atmosphericfeatures/1.1 http://xml.fmi.fi/schema/om/atmosphericfeatures/1.1/atmosphericfeatures.xsd">
  <wfs:member>
    <om:PointTimeSeriesObservation gml:id="WFS-2mM1ht29G1xf6UWShQo4Qs8lqTaJTowqYwbbpd0t.Ln15dsPTTV3c3TrvLw9NGXk6ddN03L2w70uXhh08oWliY5906pp25b
    VX8KpM0byxGNj5c61ItCnHd0mjJq4Z2XdkqaduW1V_CqTtGPZnZtunnpyc6zGLBwwaMHRry_e_1kv7.2Xl35aemHfsyxMzZh6ZefSJmbN.PDsy1qZtN.NJXdemZw1tuHxE08.mHdjy0rV0IDS24fEXhvx60
    c4Mcze25emXfow8s03L0y8uda3TLt4ZehWp15Zwt08_endnqZFHSsadhnnRd12z81Pph6ad.7n0E1uPXD509PMndm3xNbn0w9N0_du88MtaG_hl3ZMPTC30fTfyY50PXLy8390StMLNt0unW_Fzy8u2Hpp3
    7ubp13y4emjLyd0ummb17YdnXLw6eTQ6aduWn0y8JyRmh007ctrfyujVakM">
      <om:phenomenonTime>
        <gml:TimePeriod gml:id="time1-1-1">
          <gml:beginPosition>2026-10-06T19:50:00Z</gml:beginPosition>
          <gml:endPosition>2026-10-07T07:50:00Z</gml:endPosition>
        </gml:TimePeriod>
      </om:phenomenonTime>
      <om:resultTime>
        <gml:TimeInstant gml:id="time2-1-1">
```

Figure 5.10 — `curl` command output (I saved it to a file `fmi.xml`)

- Inspect and validate the received XML data with www.w3schools.com/xml/xml_validator.asp
 - Result of the check: No errors found

36. Decode this base64 encoded message with any tool(s) you prefer:

SGVsbG8gdGhlcmUgT2FtayBzdHVkZW50ISBBcmUgeW91IGhhdm1uZyBmdW4gbm93Pz8/



Decode from Base64 format
Simply enter your data then push the decode button.

SGVsbG8gdGhlcmUgT2FtayBzdHVkZW50ISBBcmUgeW91IGhhdm1uZyBmdW4gbm93Pz8/

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE > Decodes your data into the area below.

Hello there Oamk student! Are you having fun now???

Figure 5.11 — String decoded with [online tool](#)

- Decoded message: Hello there Oamk student! Are you having fun now???

37. Encode this string: "I love data processing challenges!" with base64 encoding

Encoded string: SSBsb3ZlIGRhdGEgcHJvY2Vzc2luZyBjaGFsbGVuZ2VzIQ==

Encode to Base64 format

Simply enter your data then push the encode button.

I love data processing challenges!

i To encode binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8

Destination character set.

LF (Unix)

Destination newline separator.

☐ Encode each line separately (useful for when you have multiple entries).

☐ Split lines into 76 character wide chunks (useful for MIME).

☐ Perform URL-safe encoding (uses Base64URL format).

☒ Live mode OFF

Encodes in real-time as you type or paste (supports only the UTF-8 character set).

> ENCODE <

Encodes your data into the area below.

SSBsb3ZlIGRhdGEgcHJwY2Vzc2luzYBjaGFsbGVzZVzIQ==

Figure 5.12 — String encoded with [online tool](#)

Gleb Bulygin
gbulygin@students.oamk.fi
DIN24SP
Autumn 2026

38. Describe the difference between request-response and publish-subscribe communication models

Request-response is a direct, one-to-one model: a **client** sends a request to a known **server** and waits for its answer. The client has to know the server's address, and both must be online at the same time. If the client wants new data, it must ask again (**polling**). This is how **HTTP** (REST APIs) and **CoAP** work.

```

Client — GET /temperature —> Server
Client <— 200 OK {"temp": 21.5} — Server

```

Publish-subscribe is an indirect, event-driven model with a middleman called a **broker**. **Publishers** send messages to a **topic** (e.g. `home/kitchen/temp`) without knowing who will receive them. **Subscribers** tell the broker which topics they are interested in, and the broker **pushes** each new message to them as soon as it arrives. Publishers and subscribers never talk directly. This is how **MQTT** works.

[illegible]

	Request-response	Publish-subscribe
Communication	One-to-one, client ↔ server	One-to-many / many-to-many via a broker
Coupling	Client must know the server's address	Publishers and subscribers don't know each other, only topics
Data flow	Pull — client asks when it needs data	Push — data is delivered when it is published
Timing	Synchronous: client waits for the reply	Asynchronous: no waiting for a reply
New data	Requires polling (repeated requests)	Delivered instantly on change
Scaling	Server load grows with every client request	Broker fans out one message to many subscribers
Weak point	Polling wastes bandwidth and battery	Broker is a central point of failure
Examples	HTTP/REST, CoAP, SQL queries	MQTT, AMQP, Google Pub/Sub, Kafka

For IoT, publish-subscribe is usually the better fit for **sensor data and events**: a battery-powered device just publishes a reading and goes back to sleep, and any number of apps can receive it. Request-response is better when you need a **specific answer on demand**, e.g. reading a configuration value or calling a command on a device.

39. Try MQTT websocket demo application

- Subscribe to some existing topic(s) in HiveMQ demo service
- Publish some messages to the topic(s) you subscribed

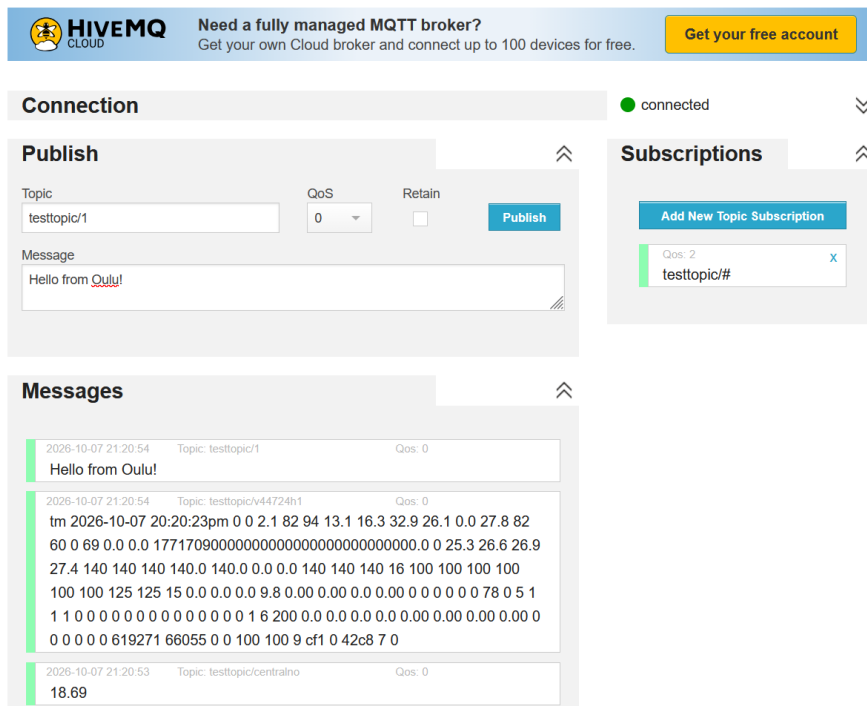


Figure 6.1 — Message "Hello from Oulu!" published to the `testtopic/1`

40. Explain what are MQTT retained messages

Normally an MQTT broker does **not store** messages: it forwards a published message to the clients that are subscribed **at that moment**, and then it's gone. A client that subscribes later has to wait for the next publish, which for a sensor reporting every 10 minutes (or a status that rarely changes) can take a long time.

A **retained message** is a normal message published with the **retain flag** set to `true`. The broker delivers it to current subscribers as usual, but it also **keeps a copy as the "last known good value"** for that topic. Whenever a new client subscribes to a matching topic, the broker **immediately sends it the retained message**, so the client knows the current state straight away.

1. Sensor publishes "21.5" to `home/kitchen/temp` (`retain = true`)
2. Broker stores "21.5" as the retained message of `home/kitchen/temp`
... time passes, no new messages ...
3. Phone app subscribes to `home/kitchen/temp`
4. Broker instantly sends "21.5" to the phone app (marked as retained)

Key rules:

- **Only one retained message per topic.** A new retained message on the same topic replaces the old one.
- **Deleting it:** publish an **empty (zero-length) payload** with `retain = true` to the topic, and the broker removes the retained message.

- **Works with wildcards:** subscribing to `home/#` delivers the retained messages of every matching topic.
- **Retained $\neq$ persistent session / queued messages.** Retained messages keep only the *latest value per topic* for *any new subscriber*. Persistent sessions (`cleanSession = false` with QoS 1/2) queue *all missed messages* for *one specific client*.
- Typical uses: device status (`online / offline` , often together with the **Last Will and Testament**), latest sensor values, and configuration values that devices read when they start.

In the HiveMQ websocket client you can try it by ticking **Retain** when publishing, then unsubscribing and subscribing again: the message is delivered again immediately.

41. List shortly some reasons why MQTT may be better than HTTP for IP-based IoT communication? (For example: HTTP vs. MQTT: A tale of two IoT protocols and MQTT Vs. HTTP: Understanding the Differences)

- **Much smaller overhead.** An MQTT fixed header is only **2 bytes**, and a small publish can fit in a few tens of bytes. An HTTP request carries text headers (method, URL, `Host` , `User-Agent` , `Content-Type` , cookies...) that are often **hundreds of bytes** — larger than the sensor reading itself. Less data means less radio time, which saves **battery and bandwidth**, and costs less on metered (e.g. cellular) links.
- **One persistent connection.** An MQTT client opens one TCP connection and keeps it open, so it doesn't repeat the TCP (and TLS) handshake for every message. With HTTP, each request either opens a new connection or has to manage keep-alive.
- **Push instead of polling.** With publish-subscribe, the broker **pushes** new data to subscribers as soon as it's published. With HTTP, a client has to keep asking the server ("any new data?"), which wastes traffic and adds delay.
- **Two-way communication.** A device behind NAT or a firewall can't easily receive HTTP requests, but because it keeps its connection to the broker open, it can still **receive commands** (e.g. subscribe to `device/42/cmd`). HTTP is client-initiated only.
- **Decoupling and one-to-many.** The sensor publishes once and the broker delivers the message to any number of subscribers (app, database, dashboard). The sensor doesn't need to know who receives it, and new consumers can be added without changing the device.
- **Delivery guarantees built in.** QoS 0/1/2 (at most once / at least once / exactly once) lets you choose reliability per message, and persistent sessions queue messages for clients that were temporarily offline. This suits **unreliable networks**.
- **IoT-specific features.** **Retained messages** give new subscribers the last known value straight away, **Last Will and Testament** tells others when a device disconnects unexpectedly, and the **keep-alive** ping detects dead connections.
- **Simple for constrained devices.** MQTT client libraries are small and run on microcontrollers (ESP32, Arduino) with very little RAM.

HTTP is still a good choice for **request-response** tasks like REST APIs, firmware downloads, and web interfaces, and it's universally supported by firewalls, proxies and tools. In practice, many IoT systems use **MQTT for device telemetry** and **HTTP for apps and APIs**.

42. What is CoAP?

CoAP (Constrained Application Protocol, RFC 7252) is basically a stripped-down HTTP for tiny devices. It keeps the familiar REST idea, so you still have resources with URIs and use GET, POST, PUT and DELETE on them, but it runs over **UDP** instead of TCP and its header is only 4 bytes. Because UDP has no delivery guarantee, CoAP adds its own: a message can be sent as *confirmable* (the receiver has to ACK it, otherwise it gets resent) or *non-confirmable* (fire and forget).

A few things HTTP doesn't have: the **Observe** option lets a client subscribe to a resource and get updates pushed to it, a bit like MQTT. It supports **multicast**, which is handy for device discovery, and large payloads can be split with block-wise transfer. The default port is 5683, or 5684 when it's secured with DTLS.

43. What is 6LoWPAN?

6LoWPAN stands for *IPv6 over Low-Power Wireless Personal Area Networks*. It's an adaptation layer that sits between IEEE 802.15.4 radios and IPv6, so that very small battery-powered devices can be real IPv6 hosts with their own addresses.

The problem it solves is size. An 802.15.4 frame is at most 127 bytes, while the IPv6 header alone takes 40 bytes and IPv6 expects packets of at least 1280 bytes. 6LoWPAN fixes this in two ways. It **compresses headers** (IPv6 + UDP can often shrink from 48 bytes to well under 10, because much of it can be derived from the link layer), and it **fragments** big IPv6 packets into several radio frames and puts them back together at the other end. Thread, which Matter devices use, is built on top of 6LoWPAN.

44. What is IETF ROLL?

ROLL is an IETF working group, short for **Routing Over Low power and Lossy networks**. It was set up around 2008 after people noticed that the usual routing protocols (OSPF, OLSR and similar) didn't suit sensor networks: they send too much control traffic, use too much memory, and expect links to be fairly stable.

The group's job was to define routing requirements for these "LLNs" (home and building automation, industrial and urban sensor networks) and then design a protocol that fits. Their main result is **RPL** (RFC 6550). They also published the Trickle algorithm (RFC 6206) and the objective functions RPL uses to pick routes.

45. Describe IETF RPL protocol?

RPL (pronounced "ripple") is the *IPv6 Routing Protocol for Low-Power and Lossy Networks*. It's a distance-vector protocol that organizes the network as a tree-like graph called a **DODAG** (Destination-Oriented Directed Acyclic Graph). The root is usually the border router that connects the sensor network to the internet.

Each node gets a **rank**, which roughly says how far it is from the root. Routes are chosen by an *objective function* that can look at hop count, link quality (ETX), energy and so on. Building the graph uses a few ICMPv6 control messages:

- **DIO**: the root and other nodes announce the DODAG and their rank, so new nodes can pick a parent
- **DIS**: a new node asks for a DIO instead of waiting for one
- **DAO**: a node advertises itself upwards so that traffic can also reach it from the root

DIOs are sent using the Trickle timer. When the network is stable, messages become rare, which saves energy, and when something changes they speed up again. RPL is optimized for sensors sending data up to the root. For downward traffic it has a *storing* mode, where nodes keep routing tables, and a *non-storing* mode, where only the root knows the routes and uses source routing.

46. Why classic computer network protocols like TCP/IP, data formats such as JSON and XML, and security systems like (PKI/HTTPS) won't usually work at all or are not very optimal to be used in resource limited wireless sensor networks (low power and lossy networks)?

A typical sensor node might have tens of kilobytes of RAM, a slow microcontroller and a battery that's supposed to last for years. On top of that, its radio frames are around 100 bytes and packets get lost all the time. The standard internet stack was designed with none of that in mind:

- **TCP** needs a handshake before any data moves, keeps state for every connection, and treats packet loss as congestion. On a lossy radio link it ends up slowing down and resending data for no real reason. Every byte sent costs battery.
- **IPv4/IPv6 headers** are big compared to the frame. Without compression, a 40-byte IPv6 header plus TCP would take most of a 127-byte frame.
- **JSON and XML** are text. `{"temperature": 21.5}` is over 20 bytes for one number that fits in two, and parsing text needs memory and CPU time the device doesn't really have. Binary formats like CBOR do the same job much more compactly.
- **PKI and HTTPS** need X.509 certificates that are often a kilobyte or more each, a TLS handshake that takes several round trips, heavy public-key maths (RSA especially), and an accurate clock to check whether certificates have expired. For a small node that's a lot of memory, energy and airtime.

That's why the constrained world uses lighter equivalents: UDP with CoAP instead of TCP with HTTP, 6LoWPAN header compression, CBOR instead of JSON, and DTLS or OSCORE with pre-shared keys or elliptic-curve keys instead of full TLS with certificate chains.

47. What is the MTU challenge for IPv4 and IPv6 over common wireless low power and lossy wireless connections (Hint: Research Zigbee/IEEE 802.15.4 and Bluetooth MTU vs IPv4 or IPv6)?

MTU (Maximum Transmission Unit) is the largest packet a link can carry in one go. The mismatch is big:

Link / protocol	Max size
IEEE 802.15.4 (Zigbee, Thread)	127-byte frame, roughly 80–100 bytes left for data after MAC headers and security
Bluetooth Low Energy	27 bytes per link-layer packet before BLE 4.2, up to 251 bytes with Data Length Extension. The default ATT MTU is only 23 bytes.
IPv6	Every link must support at least 1280 bytes (RFC 8200)
IPv4	Hosts must accept at least 576-byte packets, and Ethernet normally uses 1500

So a single minimum-size IPv6 packet doesn't fit anywhere near one radio frame. IP can't simply run on top of these links. An adaptation layer has to handle fragmentation and reassembly underneath IP: 6LoWPAN for 802.15.4, and RFC 7668 for Bluetooth LE, which uses L2CAP to do it.

Fragmentation brings its own trouble on lossy links. A 1280-byte packet turns into a dozen or more frames, and if only one of them is lost, the whole packet is lost and has to be sent again. That costs energy and airtime. The receiver also needs RAM to buffer the fragments while it waits for the rest. In practice, applications try to keep their messages small enough to fit in a single frame. This is why header compression and compact formats like CoAP and CBOR matter so much.

48. Compare and list few HTTP/1.1, HTTP/2 and HTTP/3 differences and features

	HTTP/1.1 (1997)	HTTP/2 (2015)	HTTP/3 (2022)
Transport	TCP	TCP	QUIC over UDP
Format	Plain text	Binary frames	Binary frames
Requests per connection	One at a time (pipelining exists but is barely used)	Many at once, multiplexed as streams	Many at once, streams are independent
Header compression	None	HPACK	QPACK
Encryption	Optional (HTTPS = HTTP over TLS)	Optional by spec, but browsers only use it over TLS	Always, TLS 1.3 is built into QUIC
Head-of-line blocking	Yes, at HTTP level	Fixed at HTTP level, but one lost TCP packet still stalls all streams	Gone, a lost packet only affects its own stream

A few notes on each:

- **HTTP/1.1** added persistent (keep-alive) connections and the `Host` header, which made virtual hosting possible. Because each connection handles one request at a time, browsers work around it by opening around six parallel connections per site.
- **HTTP/2** keeps the same methods, status codes and headers, but sends everything as binary frames over a single connection. It also introduced stream priorities and server push, although browsers have since dropped support for push.
- **HTTP/3** replaces TCP with QUIC. Connection setup is faster (1 round trip, or 0-RTT when reconnecting), and a connection can survive a network change, for example moving from Wi-Fi to mobile data, because it's identified by a connection ID instead of an IP address and port.

49. Use Chrome or other Chromium based browser and its developer tools (F12), and access the course web page tl.oamk.fi/iot/. From the developer tools network tab, select the main page: `iot/` and check the response headers. Answer:

- What is the connection type?
 - keep alive
- What is the server software the web server announced?
 - Apache
- Was any compression / encoding being used? (`content-encoding`)
 - gzip
- Is there `X-Xss-Protection` set in the response?
 - yes: 1; mode=block
- Is there `Strict-Transport-Security` set in the response?
 - yes: max-age=31536000; includeSubdomains;

50. What is Head-of-Line blocking challenge/problem?

Head-of-line (HOL) blocking happens when things are handled strictly in order and the first one in the queue gets stuck. Everything behind it has to wait, even if it's ready to go. It's like a supermarket queue where one customer's card doesn't work and nobody else can pay.

In HTTP it shows up at two levels:

- **HTTP level (HTTP/1.1):** a connection handles one request at a time, and responses must come back in the same order. If the first response is a big or slow one, the small ones behind it wait. Browsers work around this by opening about six connections per site, which costs extra handshakes.
- **TCP level (HTTP/2):** HTTP/2 fixes the first problem by multiplexing many streams over one connection. But TCP still delivers bytes strictly in order, so if a single packet is lost, every stream on that connection stops until the packet is resent, even streams whose data already arrived. On a lossy mobile or Wi-Fi link, this can make HTTP/2 slower than HTTP/1.1.

HTTP/3 solves this by running over QUIC, where each stream is delivered independently. A lost packet only holds up the stream it belongs to.

51. What is reverse proxy. List some advantages and features

A reverse proxy is a server that sits in front of one or more backend servers and receives client requests on their behalf. The client thinks it's talking to the website directly, but the reverse proxy decides which backend should handle the request, forwards it, and passes the response back. Common examples are **nginx**, **HAProxy**, **Traefik**, **Caddy**, and Apache with `mod_proxy`. Cloudflare-style CDNs work as reverse proxies too.

(A normal "forward" proxy is the other way round: it sits in front of the *clients* and acts on their behalf, for example a company proxy for web browsing.)

Advantages and features:

- **Load balancing:** spreads requests across several backend servers, and skips a server if it goes down
- **TLS termination:** handles HTTPS and certificates in one place, so backends can use plain HTTP on the internal network
- **Security:** hides the backend servers' real addresses and software, and gives you one place to add rate limiting, IP filtering or a WAF
- **Caching and compression:** serves static files and cached responses itself and gzips responses, which takes load off the application
- **One entry point for many services:** routes by domain or path, e.g. `/api` to one service and `/grafana` to another, all on port 443
- **Easier maintenance:** backends can be updated or swapped without clients noticing

In IoT setups this is a common pattern: nginx in front of Node-RED, Grafana and an API, all sharing one domain and one certificate.

52. What is Web application firewall (WAF). List some advantages and features

A WAF is a firewall for HTTP traffic. A normal network firewall looks at IP addresses and ports, so it allows or blocks traffic to port 443, for example. A WAF actually reads the content of HTTP requests and responses and blocks the ones that look like attacks on the web application. It's often built into a reverse proxy or CDN, e.g. **ModSecurity** with the OWASP Core Rule Set, **Cloudflare WAF**, **AWS WAF** or **Azure Application Gateway**.

Advantages and features:

- **Blocks common web attacks** like SQL injection, cross-site scripting (XSS), path traversal and remote file inclusion, roughly the OWASP Top 10
- **Virtual patching**: if a vulnerability is found in your app, a WAF rule can block exploit attempts until the code is actually fixed
- **Rate limiting and bot protection**: slows down brute-force logins, scrapers and some types of application-level DoS
- **Filtering by IP, country or reputation lists**
- **Logging and alerting**: shows what kind of attacks are hitting your site
- **Protects apps you can't easily change**, like old systems or third-party software

The downsides are that it can produce false positives, so rules need tuning, and it doesn't replace writing secure code. It's an extra layer.

53. What are Websockets?

WebSocket (RFC 6455) is a protocol that gives the browser and a server a **permanent two-way connection**. With plain HTTP, the client always has to ask first. With a WebSocket, once the connection is open, either side can send a message at any time.

It starts as a normal HTTP request with an `Upgrade: websocket` header. The server answers `101 Switching Protocols`, and from then on the same TCP connection carries small WebSocket frames instead of HTTP requests. The URLs use `ws://`, or `wss://` with TLS, and the default ports are 80 and 443, so they usually get through firewalls and proxies.

Because there's no HTTP header on every message, overhead is small and latency is low. That makes WebSockets a good fit for chat, live dashboards, online games, notifications and IoT data. The HiveMQ client in question 39 is an example: it runs **MQTT over WebSockets**, which is how a browser, which can't open a raw TCP connection, can talk to an MQTT broker.

54. What is HTTP long polling?

Long polling is a trick for getting near-real-time updates over plain HTTP, from before WebSockets were widely available.

With normal polling, the client asks "anything new?" every few seconds and usually gets "no". That wastes requests, and updates still arrive late. With **long polling**, the client sends a request and the server **doesn't answer right away**. It holds the request open until there's new data, or until a timeout of maybe 30–60 seconds. As soon as the client gets a response, it sends a new request straight away, so there's almost always one waiting on the server.

```
Client — GET /updates —> Server    (server waits...)
                                   ... new data arrives
Client <— 200 {"temp": 22} — Server
Client — GET /updates —> Server    (waits again...)
```

It works everywhere because it's just ordinary HTTP. The downsides are that every message still needs a full HTTP request with headers, the server has to keep many connections waiting, and the client has to send a new request after each message. Today WebSockets or Server-Sent Events are usually a better choice, but long polling is still used as a fallback, for example in Socket.IO.

55. Use this [tool](#) to check few websites whether the server supports HTTP/2. Two examples: www.kaleva.fi and www.oulu.fi

- www.kaleva.fi supports HTTP/2, ALPN extension is supported
- www.oulu.fi does not support HTTP/2, ALPN extension is not supported

ALPN (Application-Layer Protocol Negotiation) is the TLS extension a browser uses to agree on HTTP/2 during the TLS handshake, so a server without ALPN support can only use HTTP/1.1. I also checked both sites with `openssl s_client -alpn h2,http/1.1 : kaleva.fi negotiated h2 , oulu.fi negotiated nothing`.

The screenshot shows a web application titled "HTTP/2 Test" with the subtitle "VERIFY HTTP/2 SUPPORT". On the left, there is a sidebar with navigation links: "Web" (selected), "Website Speed Test", "Performance Test", "HTTP Header Checker", "HTTP/2 Test", and "Brotli Test". Below these are categories: "Network", "Security", and "Other". The main content area shows the URL "https://www.kaleva.fi/" entered in a text box, with a "Test" button to its right. Below the URL, the domain "www.kaleva.fi" is displayed. Two green boxes show the results: "HTTP/2 protocol is supported." and "ALPN extension is supported."

Figure 6.2 — [tool](#) result for www.kaleva.fi

Figure 6.3 — [tool](#) result for [www.oulu.fi](#)

56. Study Google Firebase documentation and advertisements. Think and list examples how to use Firebase ecosystem with Android application(s) or with some IoT other system?

Firebase is Google's Backend-as-a-Service platform. It gives you ready-made backend pieces (login, databases, file storage, push notifications, serverless functions, analytics) as SDKs for Android, iOS, web and C++. You don't have to run your own servers, and the free Spark plan is enough for small projects.

Example from my own project: Memento (Android)

I used Firebase in our group's Android app [Memento — Life in Weeks](#). The app shows your life as a grid of weeks, and you can attach notes, tags, photos and life phases to each week. It's written in Kotlin with Jetpack Compose, and it uses three Firebase products:

- **Firebase Authentication with anonymous login and account linking.** On first launch, the app quietly calls `signInAnonymously()`, so the user gets a real UID and can start saving data without registering. If they later sign in with Google (through Credential Manager) or with email and password, the anonymous account is **linked** to the new credential using `linkWithCredential()`. The UID stays the same, so no data is lost.

```
// AuthRepository.kt
try {
    // Link the anonymous account to the Google credential (data is preserved)
    auth.currentUser?.linkWithCredential(firebaseCredential)?.await()
} catch (e: FirebaseAuthUserCollisionException) {
    // This Google account already has a Firebase account – sign in normally
    auth.signInWithCredential(firebaseCredential).await()
}
```

- **Cloud Firestore for cloud backup and sync between devices.** All of a user's data lives under their own UID: `users/{uid}/notes`, `users/{uid}/tags`, `users/{uid}/phases` and `users/{uid}/data/profile`. The app writes to the local **Room** database first, so the UI stays fast and works offline, and then sends a copy to Firestore. On sign-in, the local database is rebuilt from Firestore. That's how your data follows you to a new phone.

```
// NoteRepository.kt
private suspend fun notesRef() = firestore
    .collection("users")
    .document(ensureUserId())
    .collection("notes")

notesRef().document(weekIdx.toString()).set(mapOf("note" to note)).await()
```

- **Cloud Storage for Firebase for photos.** Photos attached to a week are uploaded to `users/{uid}/photos/{week}.jpg`. When the user signs in, the app does a differential sync: it only downloads photos that are missing locally, and it removes the ones that were deleted on another device.

Firebase saved us from building and hosting our own backend, login system and file server, which wouldn't have been possible in a student project of that size.

Other ideas for Android apps

- **Cloud Messaging (FCM):** free push notifications, e.g. a reminder to write a weekly note, or an alert when something happens on the server
- **Crashlytics:** collects crash reports from users' phones with full stack traces
- **Analytics and Remote Config:** see which features people actually use, and change app behaviour or run A/B tests without publishing a new version
- **App Check:** makes sure only your real app can call your Firebase backend
- **Firebase AI Logic / ML Kit:** call Gemini models or do on-device text and image recognition from the app

IoT examples

- **Sensor data to the cloud:** an ESP32 with a temperature and humidity sensor writes readings to **Realtime Database** or **Firestore** over HTTPS or the Firebase SDK. An Android app listens to the same path and updates its charts live, because Firebase pushes changes to listening clients automatically.
- **Remote control:** the app writes `{"relay": "on"}` to `devices/{id}/commands`, and the device listens to that path and switches the relay. Device and app never need to talk directly, which is quite similar to MQTT publish-subscribe.
- **Alerts:** a **Cloud Function** triggers whenever a new reading is written. If, for example, the freezer temperature goes above -15 °C, it sends a push notification to the owner's phone with **FCM**.
- **Device and user management:** **Authentication** makes sure each user only sees their own devices, and **Security Rules** stop one device from writing into another device's data.
- **Camera or log uploads:** a Raspberry Pi camera uploads pictures to **Cloud Storage**, and the app shows them in a gallery.
- **Dashboards:** **Firebase Hosting** serves a web dashboard that reads the same database as the mobile app.

- **Long-term analysis:** Firestore data can be exported to **BigQuery** for history and statistics.

Things to keep in mind for IoT: Firebase isn't built for very high-frequency telemetry. Every write costs money once you leave the free tier, and the SDKs are too heavy for the smallest microcontrollers. A common setup is to collect data through MQTT, then store summaries or important events in Firebase, and use FCM for notifications.

57. Use hivemq.com open MQTT broker service with Python to publish MQTT messages. Use this very basic Python MQTT publish example. Change the MQTT channel name to something different if the script complaining about the authentication.

- Install Paho MQTT library to your Python development environment.

With apt install (Linux) it should be something like this: `apt install python3-paho-mqtt`

Using venv or other virtual environment with Python is strongly recommended.

With virtual Python environment it is something like (in Linux systems):

```
python3 -m venv testing
source testing/bin/activate
cd testing
pip install paho-mqtt
wget https://t1.oamk.fi/iot/dl/hivemqtt_example.txt
mv hivemqtt_example.txt hivemqtt_example.py
# ... edit the script if necessary with nano, vim or so...
python3 hivemqtt_example.py
```

- Use web browser to connect HiveMQ websocket client interface. After connecting, subscribe to `oamkiotcourse/#` channel (`#` is wildcard to receive all data)

I connected the [HiveMQ websocket client](#) and subscribed to `oamkiotcourse/#`. Because of the wildcard it received everything under `oamkiotcourse/`: the original example's `test/sensor1` and `test/sensor2` messages, my own `test/gllbhh1` and `test/gllbhh2` messages, and other students' topics such as `oamkiotcourse/student6/67` (left side of Figure 6.4).

- Modify the example Python code and publish some random data to the `oamkiotcourse` (or some channel of your own). Example code and websocket client should look something this

```
# My modifications to the script. I only have changed the messages
from paho import mqtt
import paho.mqtt.client as paho
import paho.mqtt.publish as publish

msgs = [{ 'topic': "oamkiotcourse/test/gllbhh1", 'payload': "energy: 90" }, { "topic":
"oamkiotcourse/test/gllbhh2", 'payload': "energy: 75" }]
publish.multiple(msgs, hostname="mqtt-dashboard.com", port=1883,
protocol=paho.MQTTv31)
```

- Analyse your Python MQTT client traffic with Wireshark (or with tcpdump if using some Linux server). For example, this packet capture example file is from this kind of MQTT publish message. From your Wireshark capture:
 - What is the destination IP address?
 - 3.66.250.17, one of the addresses of `mqtt-dashboard.com` (the broker hostname used in the script)
 - What are the source and destination TCP ports?
 - Source: 1761, destination: 1883. 1883 is the standard port for unencrypted MQTT. The source port is a temporary port the OS picked for this connection, so it changes on every run.
 - Can you find published data as plain text from your captured traffic sample?
 - Yes. MQTT on port 1883 isn't encrypted, so the topic (`oamkiotcourse/test/g1lbhh1`) and the payload can be read by anyone who captures the traffic. Wireshark shows the message field as hexadecimal (`656e657267793a203930`), but with right-click and `copy as ASCII` it shows correctly as `energy: 90` (Figure 6.6). Using port 8883 (MQTT over TLS) would hide it.

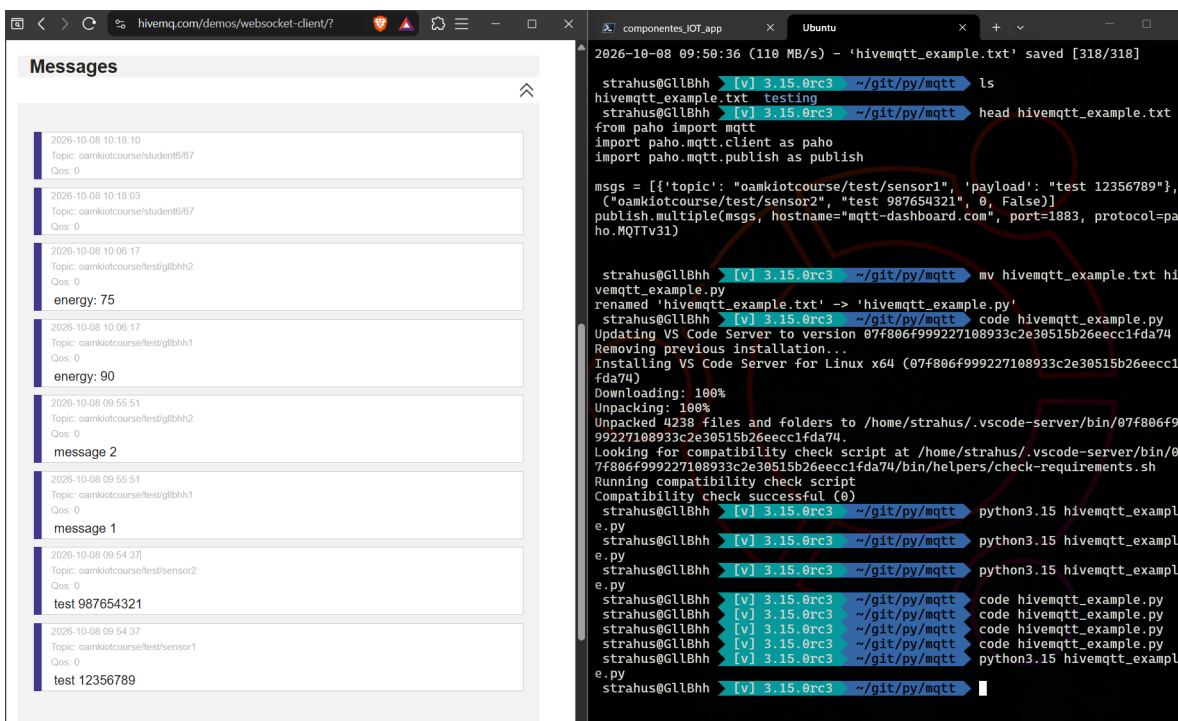


Figure 6.4 — HiveMQ websocket client subscribed to `oamkiotcourse/#` (left) receiving the messages published with the Python script (right): first the original example (`test 12356789`, `test 987654321`), then my modified messages (`message 1` / `message 2` and `energy: 90` / `energy: 75`) on `oamkiotcourse/test/g1lbhh1` and `g1lbhh2`.

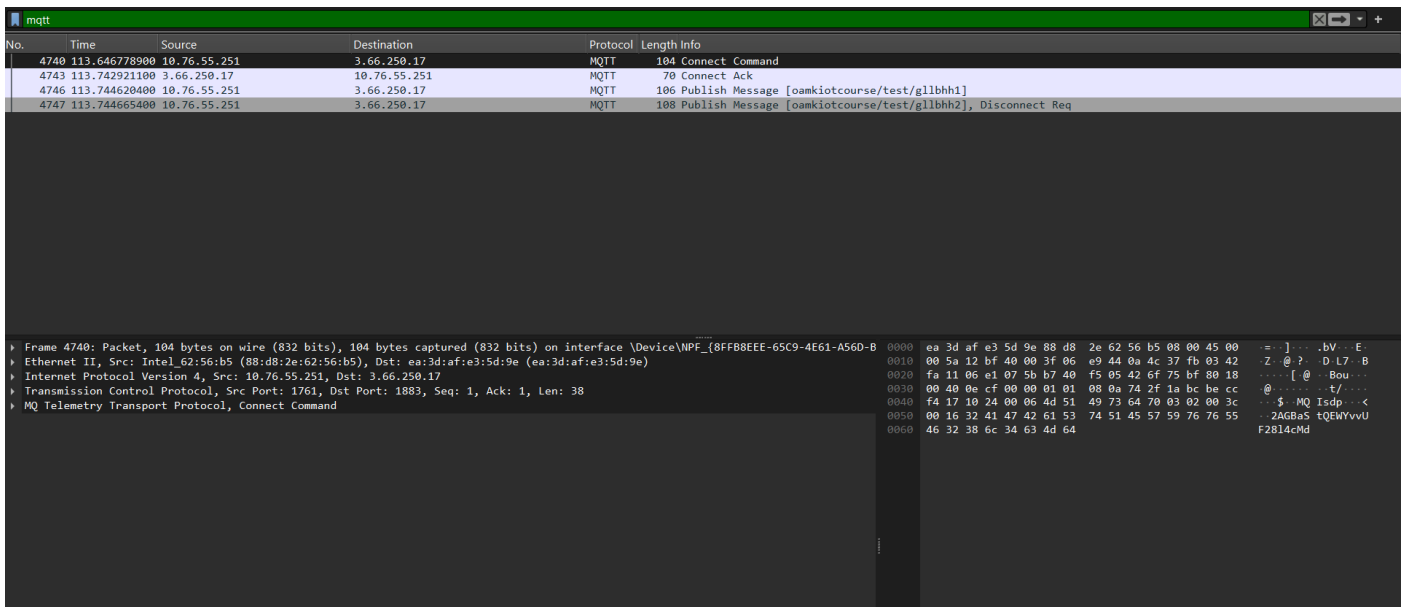


Figure 6.5 — MQTT messages published captured with wireshark

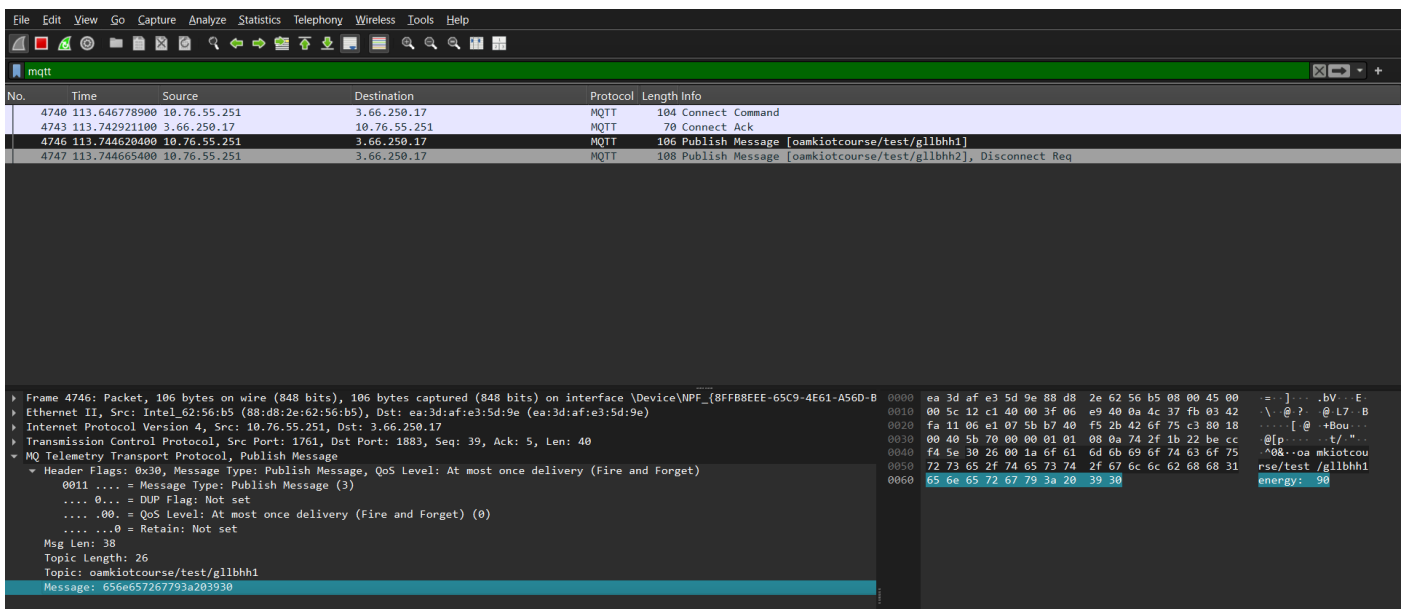


Figure 6.6 — the message shows as Hexadecimal. But with right-click and `copy as ASCII` the message shows correctly ("energy: 90")

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

Autumn 2026

Week 7

58. Describe shortly following security tools/terms/concepts:

- **CVE**

Common Vulnerabilities and Exposures. A public list where every known vulnerability gets its own ID, like `CVE-2021-44228` (Log4Shell). It's run by MITRE, so everyone (vendors, scanners, news) can refer to the same bug by the same name.

- **CVSS**

Common Vulnerability Scoring System. Gives a vulnerability a severity score from 0.0 to 10.0, based on things like how it can be reached (network or local), how hard it is to exploit, whether it needs a login or user action, and what it affects (confidentiality, integrity, availability). The score bands are Low, Medium, High (7.0–8.9) and Critical (9.0–10.0).

- **Asymmetric encryption**

Uses a key pair: a **public key** that anyone can have and a **private key** that only the owner keeps. Data encrypted with the public key can only be decrypted with the private key, and signing works the other way round. It's slow, so it's mostly used for key exchange and digital signatures. Examples: RSA, ECC.

- **Symmetric encryption**

The same secret key is used to encrypt and decrypt. It's fast and used for the actual data, but both sides need to get the key somehow, which is usually done with asymmetric encryption first. Examples: AES, ChaCha20.

- **Disassembler**

A tool that turns compiled machine code back into assembly instructions, so you can see what a program does without its source code. Used in reverse engineering and malware analysis. Examples: objdump, IDA, Ghidra.

- **Overflow vulnerability**

A program writes more data into a buffer than it has room for, and the extra data overwrites memory next to it. On the stack this can overwrite the return address and let an attacker run their own code. It's typical of C/C++ code that doesn't check input lengths. Integer overflows are a related bug where a number wraps around and leads to wrong size calculations.

- **Race condition vulnerability**

The result depends on the timing of two things happening at the same time. A classic case is TOCTOU (time-of-check to time-of-use): a program checks that a file is safe, and before it opens it, the attacker swaps the file for a link to something else.

- **ASLR/DEP/NX**

Memory protections that make overflow exploits harder:

- **ASLR** (Address Space Layout Randomization) puts the stack, heap and libraries at random addresses each run, so the attacker can't know where to jump.
- **DEP** (Data Execution Prevention, Windows name) and the **NX** bit (No-eXecute, the CPU feature behind it) mark data areas like the stack as non-executable, so injected code can't run there.

- **Ghidra**

A free, open-source reverse engineering tool released by the NSA in 2019. It disassembles and also **decompiles** binaries into C-like code, supports many CPU architectures (useful for IoT firmware), and can be scripted.

- **RCE vulnerability**

Remote Code Execution. An attacker can run their own commands or code on the target over the network. It's usually the most serious type, because it often means full control of the device or server.

- **Local privilege escalation**

An attacker who already has a normal user account on a system finds a bug or misconfiguration that gives them higher rights, usually root or Administrator. Examples are kernel bugs, badly configured `sudo` rules or SUID programs.

- **Zero-day vulnerability**

A vulnerability that the vendor doesn't know about yet, or hasn't fixed yet, so defenders have had "zero days" to patch it. Zero-days used in real attacks are very valuable and hard to defend against.

- **Zero-click exploit**

An exploit that needs no action from the victim, so no link to click and no file to open. Just receiving a message or a network packet is enough. Example: the Pegasus spyware hacked iPhones through iMessage this way.

- **SQL injection**

User input is put straight into a database query, so the attacker can change the query. For example, entering `' OR '1'='1` as a password can skip the login check, and other inputs can read or delete data. It's prevented by using parameterized queries (prepared statements).

- **Command injection vulnerability**

Like SQL injection, but the input ends up in an operating system command. If a web page runs `ping <user input>`, entering `8.8.8.8; cat /etc/passwd` makes the server run the second command too. It's very common in router and IoT web interfaces.

- **Cross-site scripting**

XSS. An attacker gets their own JavaScript into a web page that other users see, for example in a comment. The script runs in the victim's browser as if it came from the site, so it can steal session cookies or act as the user. It's prevented by escaping output and using a Content Security Policy.

- **Information disclosure**

A system reveals information it shouldn't. Examples are detailed error messages with stack traces, software version numbers in headers, exposed `.git` folders or backup files, and API responses with too much data. It's often not dangerous on its own, but it helps attackers plan the next step.

- **Code deobfuscation / obfuscation**

Obfuscation makes code deliberately hard to read without changing what it does, for example by renaming everything, encrypting strings or adding junk code. It's used to protect apps from copying and by malware to hide from analysis. **Deobfuscation** is the reverse: analysts try to make the code readable again.

- **OSINT**

Open-Source Intelligence. Collecting information from public sources like websites, social media, DNS records, job ads, GitHub and Shodan. Attackers use it to map a target before an attack, and defenders use it to see what they're exposing.

- **Data exfiltration**

Getting stolen data out of the victim's network without being noticed, for example by uploading it to cloud storage, hiding it in DNS queries, or sending it slowly in small pieces.

- **Lateral movement**

After getting into one machine, the attacker moves on to other machines in the same network, using stolen passwords, shared admin accounts or unpatched internal services, to reach more valuable systems like servers or the domain controller.

- **Command & Control**

C2 or C&C. The server an attacker uses to control compromised machines (bots) after the break-in: it sends them commands and receives stolen data. C2 traffic is often hidden inside normal-looking HTTPS or DNS traffic. The Mirai IoT botnet is a well-known example.

- **Social engineering**

Attacking people instead of technology: tricking someone into giving a password, clicking a link or doing something they shouldn't. Phishing emails, fake IT-support calls and USB sticks left in a parking lot are typical examples.

- **IDS/NIDS**

An Intrusion Detection System watches for signs of attacks and raises alerts. A **NIDS** (Network IDS) does this by monitoring network traffic, while a host-based IDS (HIDS) watches a single machine's logs and files. Detection uses known attack signatures and unusual behaviour. Examples: Snort, Suricata, Zeek. An IPS can also block the traffic, not just alert.

- **SIEM**

Security Information and Event Management. A system that collects logs from servers, firewalls, IDSs and applications in one place, connects related events together, and alerts the security team when something suspicious happens. Examples: Splunk, Microsoft Sentinel, Wazuh, Elastic Security.

59. Explain Microsoft's STRIDE threat model shortly (see the [old software vulnerability slides](#))

STRIDE is a threat modelling method from Microsoft. When you design a system, you go through each part of it (a device, an API, a data flow) and ask which of six types of threat could hit it. The name comes from the first letters, and each threat breaks one security property:

Threat	What it means	Property it breaks	IoT example
Spoofing	Pretending to be someone or something else, e.g. with stolen credentials or a fake IP address	Authentication	A fake sensor sends readings to the broker using a copied client ID
Tampering	Changing data without permission, e.g. while it travels over the network	Integrity	Someone modifies MQTT messages to change a thermostat's setpoint
Repudiation	A user can deny having done something, because there are no proper logs to prove it	Non-repudiation	No audit log shows who unlocked a smart door
Information disclosure	Private data is exposed to people who shouldn't see it, e.g. plaintext traffic or error messages	Confidentiality	Sensor data and passwords sent over unencrypted MQTT (port 1883)
Denial of service	Making a system unavailable, e.g. by flooding it with requests or crashing it with bad input	Availability	Flooding the broker or jamming the radio so the devices can't report
Elevation of privilege	A user with limited rights gets higher rights, like admin or root	Authorization	A command injection in a router's web UI gives the attacker root

The point is to find threats systematically during design, instead of relying on someone happening to think of them, and then plan a countermeasure for each one. For example: authentication against spoofing, signatures or TLS against tampering, logging against repudiation, encryption against disclosure, rate limiting against DoS, and least privilege against elevation.

60. Explain Microsoft's DREAD risk model shortly (see the [old software vulnerability slides](#))

STRIDE tells you *what kind* of threats exist. DREAD helps you decide *how serious* each one is, so you know which to fix first. Simply rating threats "high" or "low" leads to arguments, because everyone has a different gut feeling. So DREAD splits the rating into five questions:

- Damage potential: how bad is the damage if the vulnerability is exploited?
- Reproducibility: how easy is it to repeat the attack? Does it work every time, or only in rare conditions?
- Exploitability: how much skill, effort or access does the attack need?
- Affected users: roughly what share of users would be affected?
- Discoverability: how easy is it for an attacker to find the vulnerability?

Each question gets a score. In Microsoft's original guide it's 1 = low, 2 = medium, 3 = high, and many people use 0–10 instead. The scores are added together (or averaged), and the total gives the risk level. With the 1–3 scale, the total is between 5 and 15: 12–15 is high risk, 8–11 medium, and 5–7 low.

Example: an IoT device that sends its password over plain MQTT would score high on almost everything. An attacker who reads it gets full control of the device (damage), it works every time (reproducibility), only Wireshark is needed (exploitability), every device of that model is affected (affected users), and it's easy to spot (discoverability).

The weakness is that the scores are still subjective. Discoverability in particular is a problem, because it rewards hiding a bug instead of fixing it. Microsoft itself stopped using DREAD internally, and today CVSS is the more common way to rate vulnerabilities.

61. Check some CVEs of widely used applications from <https://www.cvedetails.com/> and answer:

- Describe what is the CVE scoring system

CVE itself doesn't score anything. It only gives each vulnerability an ID. The scores you see on [cvedetails.com](https://www.cvedetails.com/) come from **CVSS** (Common Vulnerability Scoring System), mostly calculated by NIST's National Vulnerability Database (NVD). The CVSS v3.1 base score is calculated from eight metrics:

- **Exploitability:** Attack Vector (network, adjacent, local, physical), Attack Complexity, Privileges Required, User Interaction
- **Scope:** whether the attack can affect other components beyond the vulnerable one
- **Impact:** effect on Confidentiality, Integrity and Availability

The result is a score from 0.0 to 10.0, plus a vector string like

`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` (= 9.8) that shows how it was calculated.

Score	Severity
0.0	None
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

A newer version, CVSS 4.0, came out in 2023. [cvedetails.com](https://www.cvedetails.com/) also shows an **EPSS** score, which estimates how likely a vulnerability is to be exploited in the wild in the next 30 days.

- When was the last time when Exim (MTA, mail transfer agent, more modern version of the application, not the Cambridge version) had a critical vulnerability? What is the CVE number?

CVE-2026-45185, published on **12 May 2026**, with a CVSS score of **9.8 (Critical)**. It's a use-after-free bug ("Dead.Letter") in how Exim handles BDAT (chunked) message bodies in builds that use GnuTLS, when the TLS connection is being shut down. An attacker who isn't logged in can send a crafted SMTP session that corrupts heap memory, which can lead to remote code execution. It affects Exim 4.97 up to 4.99.2 with STARTTLS and CHUNKING enabled, and was fixed in **Exim 4.99.3**.

Several Exim vulnerabilities have been published since then (between May and September 2026), but none of them is rated critical. The highest is CVE-2026-66140 at 8.4.

- Describe CVE-2016-6210 vulnerability shortly. Optional: How can you prevent such attack / vulnerability?

A **user enumeration** vulnerability in **OpenSSH before 7.3**, found with a timing attack. When someone tries to log in with a password, sshd hashes it. For a username that doesn't exist, it hashed a fixed fake password with the fast Blowfish algorithm. For a real user, it used the user's actual hash type (SHA-256/SHA-512 crypt), which is slower. If the attacker sends a very long password, for example 10 KB, the difference becomes clearly measurable: the server answers noticeably later when the user **exists**. This way an attacker can find out valid usernames and then focus password guessing on them.

Prevention:

- Update OpenSSH to 7.3 or newer, where the dummy hash uses the same algorithm as real users.
 - Disable password login (`PasswordAuthentication no`) and use SSH keys only. Then there's no password hashing to time.
 - Limit login attempts with `MaxAuthTries` , fail2ban or firewall rate limiting, and don't expose SSH to the whole internet if it's not needed (use a VPN or an IP allowlist).
 - In general, use the same code path and constant-time operations for valid and invalid users (see CWE-208 below).
- Describe CVE-2019-15846 vulnerability shortly

A critical **remote code execution as root** in **Exim before 4.92.2** (CVSS 9.8), published in September 2019. During the TLS handshake, the client sends the server name (SNI). If the SNI ends with a backslash followed by a null byte (`\` + `\0`), Exim's string unescaping function reads and writes past the end of the buffer, causing a heap overflow. An attacker without a login could use this to run code as root. Since Exim was used on more than half of all internet mail servers at the time, millions of servers were exposed. It was fixed in Exim 4.92.2.

- Describe CWE-208 from <https://cwe.mitre.org/data/archive.html> (download most recent PDF)

CWE-208: Observable Timing Discrepancy. Two operations take a measurably different amount of time depending on something secret, so an attacker can learn security-relevant information just by timing the responses. It's a child of CWE-203 (Observable Discrepancy).

Typical examples:

- Login takes longer for an existing user than a non-existing one. This is exactly CVE-2016-6210.
- A password or token is compared byte by byte and the comparison stops at the first wrong byte. The attacker can guess the secret one character at a time, because each correct character makes the check slightly slower.
- Cryptographic code whose timing depends on the key, which lets attackers recover the key (a side-channel attack, see question 63).

It's prevented with **constant-time** code: compare secrets with constant-time functions (e.g.

`hmac.compare_digest` in Python, `CRYPTO_memcmp` in OpenSSL), run the same code path for valid and invalid input, and use well-tested crypto libraries instead of your own implementation.

62. Study [D-Link DNS-320 ShareCenter write-up](#) in the ExploitDB

- What kind of software exploit is that?

A remote, unauthenticated backdoor exploit against a NAS device. The write-up is about the **DNS-320L** model, firmware older than 1.06, and says other ShareCenter devices may be affected too. It was published by James Bercegay / GulfTech in January 2018. It combines two vulnerabilities:

- **Hard-coded credentials (backdoor):** the firmware has a secret built-in username and password that work on every device and can't be changed by the owner.
 - **Command injection:** user input is passed to shell commands without filtering. Together they give remote code execution as root.
- Try to explain shortly (summarise) from the write-up, how the attacker can elevate access to become root (administrator) user?
 1. The CGI program `nas_sharing.cgi` contains a hard-coded login, user `mydlinkBRionyg` with password `abc12345cba` (sent base64-encoded as `YWJjMTIzNDVjYmE`). With these, the attacker gets admin access on any affected device without knowing the owner's password.
 2. In the same program, the `cmd=15` function passes a parameter straight into a shell command, so the attacker can run any command they want.
 3. The attacker turns this into a permanent web shell. First, they "log out" via `login_mgr.cgi?cmd=logout` with PHP code as the username. The name is written to a log file without filtering. Then they use the `cmd=15` injection to copy that content into `/var/www/shell.php`.
 4. Now `http://<nas>/shell.php` runs any PHP or system command they send **as root**, so the attacker has full control of the NAS and all the files stored on it.

No user interaction or real account is needed. Even the logout request works without being logged in. So any affected device reachable from the network (or internet) could be taken over.

63. Read this short [article about cracking SIM cards](#) and answer these questions:

- What is “side-channel attack”?

An attack that doesn't break the algorithm itself, but uses **physical information that leaks while the device is computing**. Examples are power consumption, electromagnetic emissions, heat, sound or timing. Because the chip's power use depends slightly on the data and key it's processing, measuring it many times and analysing the results statistically can reveal the secret key. The maths of the encryption can be perfectly secure, and the implementation still leaks the key.

- How side-channel attack was used to crack SIM cards?

At Black Hat 2015, researchers led by Yu Yu from Shanghai Jiao Tong University showed that the **AES-128 key** in 3G/4G USIM cards can be extracted with **Differential Power Analysis (DPA)**:

- They connected the SIM to a card reader and PC, and measured its **power consumption with an oscilloscope** while it ran the authentication algorithm with many different inputs. A protocol analyzer (MP300-SC2) recorded the traffic.
- By correlating the power traces with the inputs, they worked out the secret key a piece at a time.
- Cracking each of eight commercial SIM cards took only **10 to 80 minutes**.

With the key, they could **clone the SIM card**, pretend to be the owner on the mobile network and, for example, take over accounts that use SMS verification, like Alipay. The cards were vulnerable because they ran a plain software AES with no side-channel countermeasures, such as masking or adding random noise or delays.

64. Browse this [public penetration test report](#) and [news article](#) and answer these questions:

Background: in 2016, security firm MedSec and investment firm Muddy Waters published vulnerabilities in St. Jude Medical's implantable heart devices (pacemakers and defibrillators, ICDs) and the **Merlin@home** bedside transmitter that sends their data to doctors. St. Jude sued, and Bishop Fox was hired as an independent expert to check the claims. This report is that expert's report, and it confirmed the attacks work.

- Penetration test report has header *security through obscurity* (next to the item 171 and onwards). What does it mean?

Security through obscurity means relying on keeping the design secret (hidden protocols, undocumented hardware, "nobody knows how it works") as the main protection, instead of real security like authentication and encryption. It's considered bad practice, because secrets like these are eventually found through reverse engineering, and then there's nothing left protecting the system. A system should stay secure even if the attacker knows exactly how it works (Kerckhoffs's principle).

In the report, St. Jude claimed MedSec was supporting security through obscurity. MedSec had pointed out that St. Jude's use of off-the-shelf parts and the lack of anti-debugging protections made Merlin@home easy to reverse engineer. The expert disagreed (items 174–175). Well-documented standard chips with public datasheets and no anti-debugging protection simply *are* easier to analyse, and saying so is just a practical observation from reverse engineering, not an argument that hiding things is real security. The actual problems were the missing protections, like leftover debug scripts and no authentication.

- Penetration test report items 114 - 141 describe remote attack and vulnerability. What kind of problem is it?

A **battery drain attack**, a kind of **denial-of-service** attack against a medical implant. A rooted Merlin@home transmitter ran a script that repeatedly woke the implanted defibrillator over radio (2.4 GHz wake-up, then a 400 MHz session), interrogated it and disconnected, in a continuous loop. The implant **didn't authenticate** who was talking to it, so it answered every time and used battery power doing so.

Bishop Fox reproduced it:

- The target ICD lost about **3% of its battery per 24 hours**, while a control device shielded in Faraday bags lost nothing. Running non-stop, that would empty the battery in about **33 days**, or about three months if only run while the patient sleeps next to the transmitter.
- The attack also worked from **10 feet (3 m)** away, with the ICD wrapped in bacon and minced meat to simulate a human body.

St. Jude said the patient gets a low-battery alert. The expert disagreed that there is "no credible threat", because the attack is reliable, repeatable and works under real-world conditions. An implant that a patient's life depends on could stop working much earlier than expected.

65. Read this news article about [garage door security vulnerability](#) and answer:

The article is about **Nexx** smart garage door controllers, smart plugs and alarms. Security researcher **Sam Sabetan** found the problems, and there were an estimated 40,000+ devices linked to about 20,000 accounts.

- What information security and privacy issues were found and listed in the article?
 - **Anyone could open any customer's garage door** anywhere in the world, and switch Nexx smart plugs on or off, without any access to the owner's account.
 - **Privacy leak:** the attacker could see all customers' MQTT traffic, including **email addresses, names, device IDs** and the **open/close events** of garage doors. That reveals where people live and when they come and go, which is useful for burglars.
 - **API flaws (IDOR):** with just a device ID, it was possible to read another user's device history and information and change device settings.
 - Devices could be **re-registered by anyone who knew the MAC address**, which also leaked the owner's data.
 - **The vendor never responded.** Sabetan and CISA tried to contact Nexx, but the flaws were left unpatched, so CISA published an advisory and owners were told to unplug the devices.

- What was the main issue and vulnerability with MQTT configuration/architecture?

The Nexx cloud gave **every device the same universal MQTT username and password**, and that password could be found in the device firmware and in the mobile app's API traffic. All devices and apps connected to **one shared cloud broker**, and the broker had **no per-device access control (ACLs)**. So anyone logged in with the shared credentials could:

- subscribe to all topics (e.g. with the **#** wildcard), seeing every customer's messages, and
- publish to any device's topic, e.g. **replay a recorded "open" command** to open someone else's garage.

A correct design would give each device its own credentials or client certificate, and broker ACLs that only allow a device or user to access its own topics.

- Read the **CVE-2023-1748** (it's about this vulnerability). How much (i.e. how bad) is the base CVSS score? What is the CWE code for this kind of vulnerability?

The base score is **CVSS v3.1 9.3 (Critical)** from CISA (`AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:L`), and NVD's own rating is even **10.0**. The CWE is **CWE-798: Use of Hard-coded Credentials**.

66. Browse this “Secure development - towards approval” [PDF document](#) from National Cyber Security Centre Finland and answer from TESTING AND VERIFICATION chapter:

- What is unit testing?

Automated tests written and run by the developers themselves, testing small pieces of code (functions, classes) inside one component. Since developers run them all the time, bugs are found early, when they're fast and cheap to fix. Code reviews are a good place to check that unit tests exist.

- What is component testing?

Testing one component **in isolation**, with the rest of the system replaced by simulated components (mocks). These tests may be designed by a separate test team, and they should ideally be automated and run every day or night.

- What is system testing?

Testing a build of the **whole system** together. It often needs manual work, so it's slower and more expensive than unit or component tests. It can be automated, but that may need a big investment in test infrastructure.

- What is acceptance testing?

Testing done by an **independent party**, such as a separate test team, the customer or a third party, to decide whether the system is accepted. Problems found this late can be very expensive and slow to fix, because they can mean big changes and repeating the acceptance tests.

- What is static testing?

Testing **without running** the product, by inspecting things like source code and binaries. Code reviews and inspections, and automated source code analysis, are static testing. So is **software composition analysis**, which inspects a compiled binary to find out which third-party components it's built from.

- What is dynamic testing?

Testing by **running** the product and observing how it behaves, either manually or automatically. Traditionally this checks that the product meets its requirements. Today it should also test the security requirements and try to attack and abuse the product. Fuzzing and load testing (performance) are forms of dynamic testing.

- What is fuzzing?

A security-focused dynamic testing method where the product is fed **unexpected, malformed and random inputs** to find bugs. The first sign of a vulnerability is usually a crash or the product stopping responding, and with specially crafted input an attacker might even take control. Fuzzing doesn't need the source code and can be fully automated, which is also why attackers use it. The guide's advice is to fuzz your product yourself so you find the problems before others do, especially your own custom interfaces and protocols.

67. Browse this “Instructions – Supply chain attack” [PDF document](#) from National Cyber Security Centre Finland and research/answer:

- What is supply chain attack?

An attack where the attacker breaks into an organisation **through something it trusts**: a supplier, service provider, software product, update channel or open-source component. They first compromise the supplier and plant malicious code in its product. The infected product then reaches all the customers through the normal distribution channel, for example as a signed software update. Because the code comes from a trusted source, it's hard to detect, and one break-in can give the attacker a foothold in hundreds or thousands of organisations at once. From there it's used for further attacks like data theft or ransomware.

Well-known examples are **SolarWinds Orion** (2020, a backdoored update installed by around 18,000 organisations), **Kaseya VSA** (2021, ransomware pushed through an IT management tool) and the **xz Utils backdoor** (2024, planted by a fake open-source maintainer and caught just before it spread widely).

- What is 3-2-1 backup rule?

Keep at least **3 copies** of your data (the original plus two backups), on **2 different types of media or formats**, with **1 copy offline or off-site**, completely outside the network. The offline copy is the important part against ransomware and supply chain attacks, because an attacker who controls the network can't encrypt or delete it. The guide also stresses **testing the backups and practising restoring** them, because a backup you can't restore doesn't help.

- What is network segmentation and how/why it improves information security?

Dividing a network into **separate zones**, using VLANs, subnets and firewalls between them, and only allowing the traffic each zone really needs. For example, office computers, servers, guest Wi-Fi, management interfaces and IoT devices each get their own segment.

It improves security because:

- **It limits lateral movement**: if an attacker or malware gets into one device, for example a hacked IoT camera or an infected supplier tool, it can't directly reach the rest of the network. The damage stays in that segment.
- **It makes the attack surface smaller**: critical systems are only reachable from the segments that actually need them.
- **It makes detection easier**: traffic crossing between segments goes through firewalls, where it can be logged and monitored, and unusual connections stand out.
- **It makes isolation faster during an incident**: an infected segment can be cut off without shutting down everything.

For IoT this is especially important, because the devices are often weakly secured and rarely updated. Putting them in their own VLAN keeps a compromised smart device away from computers and servers.

68. Check some recent vulnerabilities being exploited in the wild from cisa.gov. Select one, summarise the problem, and search and study some news articles about the vulnerability

I picked the "MikroTrick" exploit chain in MikroTik RouterOS, because MikroTik routers are everywhere in small businesses, ISPs and IoT and industrial networks. Routers like these are a favourite target for botnets.

Item	Details
CVEs	CVE-2026-67279 (SSH authentication bypass) + CVE-2026-86060 (privilege escalation)
Added to CISA KEV	CVE-2026-86060 on 10 Sep 2026, CVE-2026-67279 on 25 Sep 2026
CWE	CWE-841 (Improper Enforcement of Behavioral Workflow), CWE-88 (Argument Injection)
CVSS	6.5 (Medium) for CVE-2026-67279 on its own, but the chain gives full admin control without a password
Found by	Sławomir Rozbicki, CERT Polska
Fixed in	RouterOS 6.49.21, 7.23.4 and 7.24.2 (released 3 Sep 2026)

The problem

1. **CVE-2026-67279**: An SSH connection normally goes in a fixed order: key exchange, then **user authentication**, then opening a session. RouterOS's SSH server could be tricked into skipping the authentication step. If the client asked for a **rekey** (a new key exchange) before logging in, the server afterwards behaved as if the login had already happened. It then accepted session channels and `exec` commands from a client that never gave a password. That's why the CWE is about not enforcing the correct order of steps (the workflow).
2. **CVE-2026-86060**: Usernames starting with a forbidden character were not handled properly when RouterOS passed them to its login process. An attacker could use this to change the **policy mask** that decides what rights the session has, and give themselves full admin rights.

Chained together, an attacker who can reach the router's SSH port gets a **full administrator console without any credentials**. CERT Polska also reported four other RouterOS bugs at the same time, for example CVE-2026-67276, where incomplete RSA key checking lets someone log in with a key without having its private part.

What the news says

- Attacks were seen **from 2 September 2026, one day before the patch** came out, so for a moment it was effectively a zero-day.
- After getting in, attackers created hidden admin accounts (often named `ops`) to keep access.
- Cybernews reported that around **122,500 MikroTik routers** with SSH reachable from the internet could be exposed. A proof-of-concept was published on GitHub soon after, which usually leads to mass scanning.
- The default RouterOS configuration blocks SSH from the internet. So the devices at risk are mostly ones where an admin opened SSH for remote management, which is common with ISPs and remote sites.

What to do

- Update RouterOS to 6.49.21 / 7.23.4 / 7.24.2 or newer.
- Don't expose SSH (or any management interface) to the internet. Allow it only from a VPN or a list of trusted IP addresses.
- Check for unknown user accounts and configuration changes. Patched versions also show a "Flagged" warning if they detect signs of compromise.
- If a router was compromised, **factory reset** it and configure it again by hand. Don't restore an old backup, because it may contain the attacker's changes.

This case connects to several earlier questions: an **authentication bypass** and **privilege escalation** (question 58), a network device used as an entry point for **lateral movement**, and a reason for **network segmentation** with management interfaces in their own protected segment (question 67).

Sources: [CISA KEV catalog](#), [CERT Polska advisory](#), [eSecurity Planet](#), [Cybernews](#), [Hexnode Threat Watch](#), [SentinelOne](#)